

CHM

Developer Handbook

Everything a new engineer needs to understand, run and safely change the Airtel Change Management platform — the React front end, the Spring Boot service and the stored-procedure layer that ties them together.

FRONT END

CHM_airtel_beta — React 19, Vite 7, TypeScript

DATA

MySQL — two schemas, stored procedures

BACK END

airtelmanagement — Spring Boot 3.4.3, Java 17

AUDIENCE

New joiners — full stack, first 30 days

About this handbook

This document was written by reading the two repositories that make up CHM, not from memory and not from a specification. Every file path, endpoint, table name, port number and configuration key printed here was taken from the source as it stands on branch `master-main` in September 2026. Where the code and an older document disagree, the code wins and this handbook follows the code.

How to read it

Parts I and II get you oriented and running — read them end to end on day one. Part III explains how the system is put together and is the part worth re-reading once you have shipped something. Part IV is a reference to the functional modules: skim it, then return to whichever chapter covers the work you are given. Part V is the house style — conventions, a full walkthrough of adding a feature, and the traps that have already cost this team time. The appendices are lookup tables.

A note on accuracy and drift

Two things in CHM move faster than any document can. The first is the set of stored procedures in the live MySQL instance, which has been hand-edited on the server in the past and can differ from the `.sql` files in the repository. The second is the RBAC grant data, which administrators change through the UI at runtime. Treat the names and shapes in this handbook as a reliable map of the *code*, and always verify the live database before you change a procedure.

Conventions used

Monospace	File paths, identifiers, HTTP routes, SQL and shell commands.
Blue boxes	Background — why something is the way it is.
Red boxes	Warnings — things that have broken builds or production before.
Green boxes	Practical tips and shortcuts.
FE / BE	Front end (<code>CHM_airtel_beta</code>) / back end (<code>airtelmanagement</code>).

Vegayan Systems — internal use only. Credentials, hostnames and API keys are deliberately omitted from this document; they live in the external configuration file described in Chapter 5.

Contents

PART I — ORIENTATION

1	What CHM Is	6
2	The Landscape: Repositories, Services, Environments	9
3	Glossary of the Domain	11

PART II — GETTING RUNNING

4	Prerequisites and Machine Setup	14
5	Running the Back End	16
6	Running the Front End	19
7	Your First Login	21

PART III — ARCHITECTURE

8	System Topology	24
9	Back-End Architecture	26
10	The Database Contract	29
11	Front-End Architecture	32
12	Authentication and Sessions	37
13	Authorization: the RBAC Model	40

PART IV — THE FUNCTIONAL MODULES

14	Module Map	45
15	CRQ Workflow: the Seven Stages	47
16	CAB Manager	51
17	Roster: View and Generation	53
18	People: Team and User Management	55
19	Configuration: Global Settings and Org Hierarchy	57
20	Inbox and Notifications	59
21	Analytics, Data Agent, SFTP and Audit	61

PART V — WORKING IN THE CODEBASE

22	Conventions	64
----	-------------------	----

23	Walkthrough: Adding a Feature End to End	66
24	Pitfalls and Troubleshooting	70
25	Build and Deployment	73

APPENDICES

A	API Endpoint Reference	76
B	Directory Map	81
C	Configuration Reference	83
D	First-Week Checklist	85

PART ONE

Orientation



Before any code makes sense you need to know what CHM is for, who uses it, and what the words on every screen mean. This part is short on purpose. Read it once, then move on to getting the thing running.

-
- 1 What CHM Is — the business problem, the users, the shape of a working day
 - 2 The Landscape — two repositories, one service, several external systems
 - 3 Glossary — CRQ, CAB, MOP, OLM ID and the rest of the vocabulary
-

01

What CHM Is

CHM is the system Airtel's network operations teams use to take a proposed change to live network infrastructure from the moment it is raised to the moment it is closed — and to staff the engineers who do the work.

The business problem

A telecom operator's network is changed constantly: a card is swapped, a software version is upgraded, a link is re-homed, a configuration is pushed. Every one of those changes carries risk — of an outage, of a service-affecting window at the wrong hour, of two teams touching the same equipment at the same time. Airtel controls that risk with a formal change-management process, and CHM is the software that runs it.

Two things happen in CHM, and understanding that they are two things is the single most useful mental model a new joiner can have:

THE TWO HALVES OF CHM

Change governance

A change request (a *CRQ*) arrives from Airtel's Remedy/Helix ITSM system. CHM walks it through seven controlled stages — validation, impact analysis, method-of-procedure authoring and review, scheduling approval, execution and closure — recording who did what, when, and with what outcome at every step.

Workforce scheduling

The same engineers who execute those changes work shifts. CHM generates and publishes monthly rosters, handles shift swaps, leave and attendance, and knows who is on duty at the hour a change is scheduled to run.

↓ the two meet here ↓

Scheduling a change against a real roster

A CRQ cannot be scheduled into a slot where the network is frozen, or where nobody with the right skills is rostered on. That constraint is why one product contains both halves rather than two products exchanging spreadsheets.

Who uses it

The roles below are the ones the code names. They are not job titles — they are RBAC role codes, and what a person sees is decided entirely by the modules granted to their role (Chapter 13).

ROLE CODE	WHAT THEY DO IN CHM
TEAM_MEMBER	An engineer. Sees their own roster and leave, works the stages of CRQs assigned to them, responds to inbox requests.
TEAM_LEAD	Runs a team. Approves swaps and leave, assigns work, sees the team's roster.
DOMAIN_HEAD , SUB_DOMAIN_HEAD	Own a slice of the organisation hierarchy. See every CRQ and every roster inside their domain or sub-domain.
FUNCTION_HEAD , VERTICAL_HEAD	Higher levels of the same hierarchy, with correspondingly broader visibility.
CAB roles	The Change Advisory Board — approvers who accept, reject, delegate or reschedule a change on behalf of the services it will affect.
SUPER_ADMIN	Configures the product itself: roles, modules, permissions, organisation hierarchy, notification rules. Also the only role that can read the audit log.

A working day, end to end

Following one change through the system is the fastest way to see how the modules relate.

1. A CRQ is raised in Remedy/Helix and lands in CHM (the platform also exposes an inbound endpoint, `/CHMDataAdapter/CreateCrq`, for that integration).
2. A planner opens **Scheduler > CRQ Workflow** and works the CRQ through *Plan & Inventory*: checking the plan, the inventory it touches and the tasks under it, and validating it.
3. *Impact Analysis* runs — partly a script on a remote host that produces a CSV of impacted circuits, partly a human judgement recorded in the UI.
4. A *MOP* (method of procedure) is created, uploaded as a PDF, then reviewed by somebody else in *MOP Validate*, who raises findings and either validates or rejects it.
5. The change goes to the **CAB** for approval. Each affected service has an approver; they approve, reject, delegate, or ask for a reschedule.
6. Approved, the CRQ is *scheduled* into a slot — checked against network freeze windows, holidays, and the roster of who is actually on duty.
7. On the night, *Activity Implement* tracks execution ring by ring; *Closure* records the outcome and pushes status back to Remedy and to Cygnet.
8. Every state change along the way can raise a notification into somebody's **Inbox**, and every state-changing action by a user is written to the **audit log**.

WHY THIS MATTERS FOR YOUR CODE

Almost every screen in CHM is a view onto one of two record sets: a CRQ and its stage history, or a roster and its assignments. When you are lost in an unfamiliar feature, ask which of the two it is reading, and the data flow usually becomes obvious.

What CHM is not

- **Not the system of record for the change itself.** Remedy/Helix is. CHM enriches, schedules and governs; it pushes status back.
- **Not an ORM application.** There is no JPA, no Hibernate, no entity graph. All data access is stored procedures called through `JdbcTemplate` (Chapter 10).
- **Not a microservice estate.** One Spring Boot deployable, one React bundle, one reverse proxy.

02

The Landscape

Two repositories, one runtime service, one browser bundle, two MySQL schemas and a handful of external systems reached over HTTP and SSH. Here is the whole map on one page.

The repositories

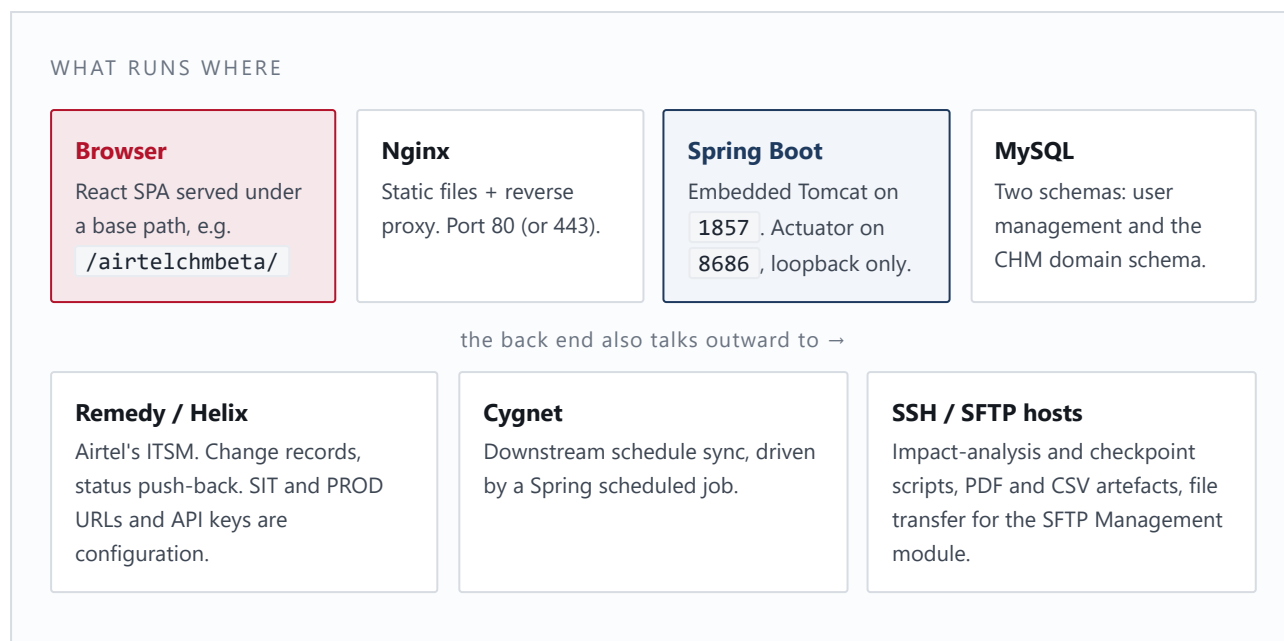
REPOSITORY	CONTENTS
CHM_airtel_beta the front end	React 19 + TypeScript SPA built with Vite 7. Also carries the <code>deployment/</code> package — Nginx configs, systemd units, Windows and Linux deployment guides — for the <i>whole</i> product, front and back.
airtelmanagement the back end	Spring Boot 3.4.3 on Java 17, packaged as a WAR with an embedded Tomcat. Also carries <code>db/migration/</code> — one dated <code>.sql</code> file per schema change, committed alongside the code that needs it.

Both repositories sit under `D:\CHM_airtel_scheduler\Github_Repo\` on a standard developer machine. A third, read-only repository — the legacy `changemanagement` back end — is kept around as the authoritative source when porting an old endpoint forward.

BOTH REPOSITORIES AUTO-COMMIT

A watcher process commits (and sometimes merges) changes in both repositories within seconds of a file being saved. It is not run by you and it is not run by your editor. The practical consequences: `git status` is often clean when you expect it to be dirty, so check `git log` instead; and never assume an uncommitted experiment will stay uncommitted.

The runtime pieces



Environments

ENVIRONMENT	NOTES
Local dev	Vite dev server on <code>5174</code> (host <code>0.0.0.0</code> , so a colleague on the LAN can open it); the back end run from IntelliJ on <code>1857</code> . The API base URL is set per-developer in <code>.env.local</code> .
SIT	<code>remedy.env=sit</code> in <code>application.properties</code> selects the SIT Remedy URL and credentials. This is the default in the repository.
Production	Nginx fronting everything on one origin, so CORS never applies. Front end built with <code>.env.production</code> ; back end started with overrides for the Windows-specific paths baked into the WAR.

ONE ORIGIN IN PRODUCTION, MANY ORIGINS IN DEV

In production the browser only ever talks to Nginx, which serves the SPA and proxies the API, so front end and API share an origin and the CORS allow-list in `SecurityConfig.java` is irrelevant. In development they are different origins, and that hard-coded allow-list of developer IP addresses *is* what lets your machine call the shared back end. If your requests fail with a CORS error, your IP is probably not in the list — see Chapter 24.

Branches and history

The working branch in both repositories is `master-main` ; `main` is the branch pull requests target. At the time of writing the front-end repository carries around 316 commits from two active contributors. Commit messages are terse and feature-shaped (“Add MOP validation workspace and PDF viewer”), and the dated migration file in `db/migration/` is normally the best index of when a behaviour changed.

03

Glossary of the Domain

CHM's vocabulary is telecom-operations vocabulary. You will not guess these from the code, and the code assumes you know them. This is the chapter to dog-ear.

Change management

TERM	MEANING
CRQ	Change Request. The central record. Identified by a <code>crqNo</code> (a human-facing string) and a <code>crqId</code> . Its authoritative state lives in <code>CRQ_MASTER_TBL</code> : <code>current_stage</code> plus <code>current_status</code> .
Stage	One of the seven controlled steps a CRQ passes through. The database enum values are <code>VALIDATE</code> , <code>IMPACT_ANALYSIS</code> , <code>MOP_CREATION</code> , <code>MOP_VALIDATION</code> , <code>SCHEDULING_APPROVAL</code> , <code>EXECUTION</code> , <code>CLOSURE</code> .
MOP	Method of Procedure. The step-by-step document describing exactly how the change will be executed and rolled back. Authored in <i>MOP Create</i> , reviewed in <i>MOP Validate</i> , stored as a PDF.
CAB	Change Advisory Board. The approval body. In CHM, a CAB session is a scheduled meeting; a CAB approver holds approval authority for one or more <i>services</i> .
Service approval	One CRQ affects several services; each gets its own approval record, keyed by <code>serviceApprovalId</code> . This is why CAB endpoints take a service approval id rather than a CRQ id.
Impact analysis	Determining which circuits and customers a change will affect. Partly a remote Python script producing a CSV batch, partly a recorded human decision.
Checkpoint	Pre- and post-change health snapshots pulled from network equipment, fetched over SFTP and summarised in the CRQ Review screen.
Ring	An execution grouping used during Activity Implement — work proceeds ring by ring and can be blocked at a ring.
Network freeze	A configured window during which no change may be scheduled. Managed in Global Settings alongside the holiday calendar.
Remedy / Helix	Airtel's ITSM platform, the system of record for change tickets. CHM reads from it and pushes status back to it.
Cygnnet	A downstream system that receives schedule information. Synchronised by a periodic job (<code>remedy.cygnnet-sync.*</code> properties).
SA / NSA	Service Affecting / Non Service Affecting — the change-impact classification carried on a plan.

Organisation and people

TERM	MEANING
OLM ID	The employee identifier used as the login username throughout CHM. It appears in the code as <code>oId</code> and is what you type on the login screen.
Org hierarchy	Four levels, in order: <i>Vertical</i> → <i>Function</i> → <i>Domain</i> → <i>Sub-domain</i> . Nearly every listing endpoint is scoped by domain and sub-domain, and the filter bar at the top of most screens is choosing a position in this tree.
Team	A group of engineers under a sub-domain. Team Management is where people are created, edited, activated and bulk-uploaded.
Roster	The published shift schedule. <i>Roster View</i> shows it; <i>Roster Generation</i> produces it.
Golden set	A validated baseline shift pattern used as the starting point for generating a new roster.
Future week	The forward-looking part of the roster being generated, before it is published.
Shift swap / shift change	Two distinct request types. A swap is between two engineers and needs both a peer and a manager decision; a change is a manager reassigning a shift.

Platform vocabulary

TERM	MEANING
Module / sub-module / permission	The three levels of the RBAC model. A role is granted permissions on sub-modules; a module is visible if any of its sub-modules carry a grant. Stored in the <code>WEB_*</code> tables.
<code>WEB_*</code> tables	The live RBAC store: <code>WEB_MODULE</code> , <code>WEB_SUB_MODULE</code> , and the role/permission mapping tables. An older <code>NOTIF_*</code> set exists and is <i>not</i> the source of truth for the UI.
Nav registry	<code>src/rbac/navRegistry.tsx</code> — the single declaration of every sidebar item, the module or sub-module it requires, and the URLs it covers. The sidebar, the route guard and the default-landing logic all read it.
<code>AUTH_JWT_TOKENS</code>	The session table. One live row per user; the JWT's <code>jti</code> is the key. If the row is gone or expired, every request answers "Invalid session" regardless of how healthy the JWT looks.
Procedure	A MySQL stored procedure. In CHM this is the API between Java and the data — there is essentially no hand-written SQL in feature services.
DB1 / DB2	The two data sources, labelled that way in the SQL log. DB1 is the user-management schema (<code>jdbcTemplateOne</code> , the primary); DB2 is the CHM domain schema (<code>jdbcTemplateTwo</code>).

PART TWO

Getting Running



A CHM checkout does not run out of the box, and the two reasons it does not are both worth understanding rather than working around: the back end refuses to start without an external configuration file it does not ship, and it silently miscompiles on the wrong JDK. This part gets you from a clean machine to a browser tab showing the login screen.

-
- | | |
|---|--|
| 4 | Prerequisites and machine setup — including the JDK trap |
|---|--|
-
- | | |
|---|--|
| 5 | Running the back end — the external config file, ports, logs |
|---|--|
-
- | | |
|---|--|
| 6 | Running the front end — env files, scripts, dev server |
|---|--|
-
- | | |
|---|---|
| 7 | Your first login — sessions, test accounts, what to click |
|---|---|

04

Prerequisites and Machine Setup

Four pieces of software, one of which has a version requirement that will waste a morning if you get it wrong.

What to install

TOOL	VERSION	WHY
JDK	17	Build and run the back end. See the warning below — this is not negotiable.
Node.js	20 or 22 LTS	Front-end build. The reference machine runs 22.18.
MySQL client	8.x	You will read and edit stored procedures constantly. A GUI is fine, but a CLI client is what the team uses.
IntelliJ IDEA	current	How the back end is run in development. VS Code is used for the front end.
Git	current	Both repositories.

THE JDK TRAP — READ THIS BEFORE YOUR FIRST BUILD

The back end must be built and run on **JDK 17**. On a newer JDK (25, for example) Lombok's annotation processing silently stops generating getters and setters, and the compiler then reports hundreds of “cannot find symbol” errors across roughly a hundred `@Data`-annotated classes. The errors point at your model classes and look like the code is broken; it is not.

Set `JAVA_HOME` to a JDK 17 installation and confirm with `java -version` before you conclude anything is wrong with the repository. In IntelliJ, check both *Project SDK* and the Maven runner JDK.

Clone the repositories

```
D:\CHM_airtel_scheduler\Github_Repo\
├─ CHM_airtel_beta\      # front end (this is also where deployment/ lives)
└─ airtelmanagement\    # back end (this is also where db/migration/ lives)
```

Keep the layout above. Several team scripts, and every path in this handbook, assume it.

Database access

CHM does not create its own schema on startup — there is no Flyway, no Liquibase and no Hibernate DDL. You connect to an existing MySQL instance with the CHM schemas already loaded. Ask the team for:

- The host, user and password for the two schemas (user management and the CHM domain schema).

- Whether you are pointing at a shared development database or a local copy. Most developers share one, which is why editing a stored procedure affects everybody — see Chapter 10.

Verify before you go further

- `java -version` reports 17.
- `node -v` reports 20 or 22.
- You can connect to MySQL and run `SHOW PROCEDURE STATUS WHERE Db = '<chm schema>';` and see a long list. If that list is short or empty, you are pointed at the wrong schema.

05

Running the Back End

The service reads its database credentials, SSH hosts and SFTP paths from a file outside the repository. Without that file it will not start, and the failure is not obvious. Create it first.

Step 1 — the external configuration file

`AppPropertiesConfig` is constructed before anything else needs a data source. In its constructor it picks a path based on the operating system, opens it, and reads every key it needs:

OS	PATH (HARD-CODED, NOT OVERRIDABLE)
Windows	<code>C:\vegayan\simplus\airtelcms-config.properties</code>
Linux	<code>/home/vegayan/simplus/config_airtel.properties</code>

Any other operating system throws `Unsupported operating system detected` at startup.

A template ships in the front-end repository at `deployment/config/airtelcms-config.sample.properties`. Copy it to the path above and fill it in. The keys fall into four groups:

GROUP	KEYS
User-management DB (DB1)	<code>DBSOURCE_USERMGMT_IP</code> , <code>_USER</code> , <code>_PASS</code> , <code>_DBNAME</code>
CHM domain DB (DB2)	<code>DBSOURCE1_IP</code> , <code>_USER</code> , <code>_PASS</code> , <code>_DBNAME</code>
SSH hosts	<code>SSH_*</code> and <code>SSH2_*</code> — host, port, credentials and the script paths for impact analysis, checkpoint refetch, shift-generation validation and PDF retrieval
SFTP	<code>SFTP_HOST</code> , <code>SFTP_PORT</code> , credentials, and the remote directories for raw JSON, CRQ work logs, batch CSVs and impact Excel files

YOU DO NOT NEED EVERY KEY TO START

Only the two database groups are required for the application to come up and for most screens to work. The SSH and SFTP keys matter for impact analysis, checkpoints, MOP PDFs and the SFTP Management module; leave them blank until you work on those, and expect those specific features to fail rather than the whole service.

Step 2 — what lives in `application.properties` instead

Everything that is not a secret-per-environment sits in `src/main/resources/application.properties` and is committed:

PROPERTY	VALUE / MEANING
<code>server.port</code>	<code>1857</code> — the API port.
<code>management.server.port</code>	<code>8686</code> — Actuator, on its own port. Keep it off the proxy.
<code>jwt.token.expirationTime</code>	<code>604800000</code> ms — seven days.
<code>jwt.secret</code> , <code>auth.master.password</code>	Signing secret and the master password used for test logins.
<code>remedy.env</code>	<code>sit</code> or <code>prod</code> — selects which Remedy URL and API keys are used.
<code>remedy.cygnnet-sync.*</code>	Enable flag, fixed delay and a separate <code>push-enabled</code> flag. Both are <code>false</code> in the repository; <code>push-enabled</code> must be <code>true</code> in production.
<code>spring.task.scheduling.pool-size</code>	<code>3</code> — the Cygnnet sync, the session cleanup job and the Excel upload job store each need a thread.
<code>spring.servlet.multipart.max-* size</code>	<code>100MB</code> — Excel and PDF uploads.
<code>analytics.reports.base-path</code>	Folder the Analytics > Reports file browser reads generated <code>.xlsx</code> reports from.

SECRETS ARE COMMITTED TODAY

The JWT signing secret, the Remedy passwords and several API keys are currently plain text in `application.properties`, which is in git. This is a known finding, recorded in the deployment README with the recommendation to move them to the external config file. Do not add new secrets to this file, and do not paste its contents into tickets, chat or documents.

Step 3 — run it

In development the team runs the application from IntelliJ, using the `AirtelmanagementApplication` main class. From the command line:

```
cd D:\CHM_airtel_scheduler\Github_Repo\airtelmanagement
mvnw.cmd clean package          # builds target\changemanagement-test.war
java -jar target\changemanagement-test.war
```

The WAR is executable — `spring-boot-starter-tomcat` is declared `provided` in the POM but the Spring Boot Maven plugin repackages it with an embedded container, and the deployment guides run it with plain `java -jar`.

Confirm it is up

```
curl http://127.0.0.1:8686/actuator/health
curl http://127.0.0.1:1857/test/hello
```

`/test/**` is a public endpoint (no token required), which makes `/test/hello` the cheapest possible reachability check.

THE DEV BACK END DOES NOT HOT-RELOAD

There is no dev-tools restart configured. After changing back-end code you must stop and restart the application in IntelliJ — a surprising number of “my endpoint returns 404” reports end here.

Logs

Logging is configured in `logback-spring.xml`. Two things are worth knowing on day one:

- Both `JdbcTemplate` s are `LoggingJdbcTemplate` s, so **every SQL statement is logged as a runnable line with its arguments already substituted**, prefixed `DB1` or `DB2`. When a screen shows the wrong data, copy the logged `CALL` into a MySQL client and run it — that single habit resolves most data bugs.
- Several named loggers exist for noisy flows: `Cancel_Crq_Logger`, `Submit_Plan_External_Logger`, `CRQ_Update_To_Chm` and `Schedule_Reschedule_Crq`. They are declared on `BaseService`, so any service can write to them.
- The log directory is `D:/CHM_LOGS` on Windows. On Linux this is overridden at launch by pointing `--logging.config` at `deployment/config/logback-linux.xml`.

06

Running the Front End

Two env files decide which back end you talk to and what URL path the app lives under. Both matter, and the npm scripts have names you would not guess.

The scripts

COMMAND	WHAT IT DOES
<code>npm run chalu-hoja</code>	<code>vite</code> — the dev server. Binds <code>0.0.0.0:5174</code> and opens a browser.
<code>npm run ready-hoja</code>	<code>vite build</code> — production build into <code>dist/</code> .
<code>npm run lint</code>	ESLint across the project.
<code>npm run preview</code>	Serves the built <code>dist/</code> locally — useful for reproducing a bug that only appears in a production build.

Yes, the dev and build scripts really are called *chalu-hoja* and *ready-hoja*. Nothing depends on the names; they are just what the team uses.

Environment files

Vite loads `.env`, then `.env.[mode]`, then their `.local` variants, with later files winning. Three files matter:

FILE	ROLE
<code>.env</code>	Committed development defaults. Contains a list of commented-out back-end IP addresses — the team's history of who was hosting the API.
<code>.env.local</code>	Your file. Not shared. Put your own <code>VITE_REACT_APP_BASE_URL</code> here rather than editing <code>.env</code> and creating a conflict for everyone.
<code>.env.production</code>	Used automatically by <code>vite build</code> . Points at the deployed back end and sets the deployment base path.

The two variables you will actually set

VITE_REACT_APP_BASE_URL	Base URL of the Spring Boot API. Every RTK Query request is relative to it. For a local back end this is <code>http://localhost:1857</code> ; for a colleague's, their LAN IP and <code>1857</code> .
VITE_APP_BASE_PATH	The URL sub-path the SPA is served from, e.g. <code>/airtelchmbeta/</code> . It feeds Vite's <code>base</code> , and <code>BrowserRouter</code> 's <code>basename</code> in <code>App.tsx</code> derives from that same value at runtime — so the two can never drift. Retargeting a deployment to a different path is a config change, never a code change.

CORS IN DEVELOPMENT

If you run the back end on your own machine and the front end on the dev server, you are on two origins. The allowed origins are a hard-coded list in `SecurityConfig.corsConfigurationSource()` — a set of developer LAN addresses on ports 5173–5178 plus the production host. If your machine's address and port are not in that list, every request fails in the browser with a CORS error while working fine in `curl`. Add your origin to the list (and mention it in your PR — the list is long precisely because everybody does this).

Start it

```
cd D:\CHM_airtel_scheduler\Github_Repo\CHM_airtel_beta
npm install
npm run chalu-hoja
```

The dev server prints a local and a network URL. Because `host` is `0.0.0.0`, the network URL genuinely works from another machine on the LAN — handy for showing work in progress, and the reason so many IPs are in the CORS list.

What you will see build-wise

Two Vite settings in `vite.config.ts` exist to solve real bugs and are explained fully in Chapter 24; know that they are deliberate:

- `resolve.dedupe` and `optimizeDeps.include` guarantee a single copy of React, MUI and Emotion. Without them you get an “Invalid hook call” crash that looks like a hooks bug and is not.
- A `manualChunks` function puts nearly all of `node_modules` into one `vendor` chunk, with `pdfjs` split out. This is a request-count fix: the default splitting produced 146 JS files, many under 300 bytes, and the burst of requests tripped rate limiting.

07

Your First Login

CHM allows exactly one live session per account. That single design decision explains most of the confusion new joiners hit in their first hour.

How signing in works

1. You enter an **OLM ID** and password. The front end posts them to `POST /auth/v1/signin`.
2. The back end looks the username up in `AUTH_CREDENTIAL`, checks the password, and then checks whether that account already has a live session.
3. On success it mints a JWT whose subject is the numeric user id and whose `jti` is a fresh UUID, writes a row to `AUTH_JWT_TOKENS`, returns the token as `accessToken`, and also sets it as an `HttpOnly` cookie named `jwt`.
4. The front end stores the token, then calls `GET /users/v2/getloggeduserdetails` to fetch the profile *and* the full module → sub-module → permission tree that drives the whole UI.
5. You land on the first route your permissions allow — there is no fixed home page.

The three failure messages

MESSAGE	HTTP	MEANING AND FIX
User Not Found	404	No such OLM ID. Note that accounts do get deleted from the shared dev database — an ID that worked last month may not exist today.
Invalid Password	401	Wrong password. A master password configured in <code>auth.master.password</code> works for any account in non-production environments.
Already Logged	403	That account has a live session — possibly yours, in another browser or on another machine, possibly a colleague's. See below.

“ALREADY LOGGED” AND HOW TO CLEAR IT PROPERLY

The login screen offers to terminate the stranded session. That calls `POST /auth/v1/session/terminate`, which is deliberately reachable without a token — you cannot get one while the old session holds the slot — but **re-checks the account password** before ending anything. This replaced an older design where posting a bare `{ "olmId": "..."}` to the logout route would end anyone's session if you knew their ID.

The blunt alternative, when you have database access, is to delete the account's row from `AUTH_JWT_TOKENS`. That is the same thing the endpoint does.

Session behaviour you should expect

- **Refreshing the page does not log you out.** The token lives in `localStorage`, and on every cold start `AuthHydrator` re-validates it against the back end before trusting it.
- **Opening a second tab works.** `localStorage` is shared across tabs, and a `storage` event keeps them in sync — sign out in one tab and every other tab signs out too.
- **“Invalid session” on every request** means the `AUTH_JWT_TOKENS` row is gone or expired. It never means the JWT itself was malformed. See Chapter 24.
- **A 401 anywhere tears the session down deliberately.** The RTK Query base query clears storage, dispatches `logout`, resets the entire query cache and shows a toast — so the next user to sign in on that machine can never see the previous one's cached data.

What to click first

Once in, spend twenty minutes doing this, in order. It maps the product onto the chapters ahead faster than reading does:

1. **Scheduler › CRQ Workflow.** Open a CRQ. Look at the stage rail across the top — that is the seven-stage model of Chapter 15 rendered directly.
2. **Scheduler › CRQ Journey.** The same CRQ as a pipeline. Note that this view is shared with the CAB module.
3. **Roster › Roster View.** A month of shifts for a sub-domain. Click the small *i* on a cell for the daily assignment detail.
4. **My Dashboard.** Your own attendance, roster, leave and notification preferences — the merged Home + Me workspace.
5. **Global Settings › Admin Settings** (if your role can see it). This is the RBAC editor. Everything you change here changes what other people's sidebars look like.

IF THE SIDEBAR IS NEARLY EMPTY

That is RBAC working, not a bug. Sidebar items are driven entirely by the modules granted to your role. Ask for a role with broader grants, or read Chapter 13 to see exactly which module name each item requires.

PART THREE

Architecture



CHM is conventional in its choice of frameworks and unconventional in exactly two places: all data access goes through stored procedures, and all UI visibility goes through one nav registry. Understand those two and the rest of the system follows normal Spring and React idioms.

-
- | | |
|---|---|
| 8 | System topology — what talks to what, over which port |
|---|---|
-
- | | |
|---|---|
| 9 | Back-end architecture — packages, layering, BaseService, errors |
|---|---|
-
- | | |
|----|--|
| 10 | The database contract — procedures as the interface, and its rules |
|----|--|
-
- | | |
|----|---|
| 11 | Front-end architecture — features, one RTK Query API, routing, bundling |
|----|---|
-
- | | |
|----|---|
| 12 | Authentication and sessions — JWT plus a database session row |
|----|---|
-
- | | |
|----|---|
| 13 | Authorization — the RBAC model and the single registry that enforces it |
|----|---|

08

System Topology

One browser bundle, one reverse proxy, one Java process, two schemas and a set of outbound integrations. Nothing is distributed; nothing is queued.

Production topology

```

Browser
├── HTTP :80 (or :443 – see deployment/HTTPS-MIGRATION.md)
└──
  ▼
  Nginx
  ├── /airtelchmbeta/      → static React build (dist/), SPA fallback to index.html
  ├── /changemanagementnew/ → proxy_pass → 127.0.0.1:1857 (Spring Boot)
  └── /                    → redirect → /airtelchmbeta/

Spring Boot (embedded Tomcat, :1857; Actuator :8686, loopback only)
├── MySQL DB1 user management schema (jdbcTemplateOne, @Primary)
├── MySQL DB2 CHM domain schema (jdbcTemplateTwo)
├── Remedy / Helix HTTPS change records, status push-back
├── Cygnet HTTPS schedule sync (scheduled job, disabled by default)
├── SSH host(s) :22 impact analysis, checkpoint refetch, CSV/PDF scripts
└── SFTP host(s) :22 MOP PDFs, raw JSON, work logs, batch CSVs

```

WHY IT IS SINGLE-ORIGIN

In production the browser only ever contacts Nginx. The SPA's API base URL is a path on the same host, so front end and API share an origin and the browser never performs a CORS preflight. That is why the hard-coded CORS allow-list in `SecurityConfig` is harmless in production and essential in development.

The two data sources

`MultiDataSourceConfig` builds two HikariCP pools from the external config file and wraps each in a `LoggingJdbcTemplate` labelled `DB1` or `DB2`:

BEAN	SCHEMA	USED FOR
<code>jdbcTemplateOne</code> @Primary	User management	Credentials, sessions (<code>AUTH_JWT_TOKENS</code>), users and roles, RBAC (<code>WEB_*</code>), audit log, organisation hierarchy.
<code>jdbcTemplateTwo</code>	CHM domain	CRQs and their stages, plans and activities, rosters, CAB, notifications.

Both pools are sized at 50 connections with a 30-second leak-detection threshold, and both are validated at startup — a bad credential in the external config file fails loudly at boot rather than on the first request.

Picking the wrong template is a common early mistake and produces “*PROCEDURE does not exist*”. The same procedure name can legitimately exist in both schemas, which is exactly why the log lines carry the DB1/DB2 label.

Scheduled work inside the process

JOB	WHAT IT DOES
<code>SessionCleanupJob</code>	Purges expired rows from <code>AUTH_JWT_TOKENS</code> using the same “live session” predicate the login guard and the token validator use.
<code>RemedyCygnetSyncScheduler</code>	Periodically pushes schedule data to Cygnet. Delay is measured from the end of the previous cycle so a slow batch delays rather than overlaps. Disabled in the repository.
<code>ExcelUploadJobStore</code>	Backs the asynchronous employee bulk upload — an in-memory job store, polled by the UI for status.

Spring's scheduler defaults to a single thread; the pool is explicitly set to three so these three cannot block each other.

Request lifecycle

```

HTTP request
→ CorsFilter                allow-list from SecurityConfig
→ JwtAuthenticationFilter    skipped entirely for PublicEndpoints.PATTERNS
    └─ extract token: Authorization: Bearer ..., else the httpOnly `jwt` cookie
    └─ verify signature + expiry      (JwtUtil)
    └─ verify the session row        (TokenValidationService → AUTH_JWT_TOKENS)
    └─ load authorities only when the handler carries @PreAuthorize
→ Spring Security authorization (authenticated() / @PreAuthorize)
→ @RestController method
→ @Service extends BaseService
→ DatabaseUtils → JdbcTemplate → CALL some_procedure(...)
→ DTO ← BeanPropertyRowMapper
→ ResponseEntity (errors normalised by GlobalExceptionHandler)

```

A CHEAP WAY TO LEARN THE SYSTEM

Pick any screen, open the browser network tab, note the request path, then grep the back end for that path. You will land in a controller whose service is usually under 300 lines and whose entire job is to call one or two procedures. That round trip — screen to controller to procedure — is the shape of nearly every feature in CHM.

09

Back-End Architecture

Package by feature, three classes per feature, one shared base class and one shared utility.

There is no repository layer in the usual sense, because the procedures are the repository.

Package layout

Everything lives under `com.vegayan.airtelmanagement`. Each functional area is a top-level package containing the same three or four sub-packages:

```
com.vegayan.airtelmanagement
├── common/           shared infrastructure – read this first
│   ├── config/      MultiDataSourceConfig, AppPropertiesConfig, WebClientConfig, ...
│   ├── security/    SecurityConfig, PublicEndpoints, JWT filter, session validation
│   ├── service/     BaseService, CommonService, LoggingAspect
│   ├── util/        DatabaseUtils, PaginationUtils, LoggingJdbcTemplate, ...
│   ├── dto/         ApiResponse, DbResponse, PageResponseDto, ProcedurePageResult
│   └── exception/   GlobalExceptionHandler + the exception types it maps
├── auth/ user/ usermanagement/ teammanagement/ people and sessions
├── scheduler/ cabmanager/ remedy/ sygnet/ change management
├── rostersview/ rostersgeneration/ attendance/ me/ scheduling and self-service
├── globalsettings/ orghierarchy/ notification/ configuration
├── analyticsreport/ crqanalytic/ dataagent/ reporting
├── audit/ impactbatch/ filetransfer/ cross-cutting and integrations
└── activity/ attributeupdate/ dashboard/ ...
```

each feature package:

```
controller/ @RestController – HTTP shape only
service/    @Service extends BaseService – the procedure calls
dto/        request and response records/classes
repository/ present in a few packages where raw queries were unavoidable
```

NOTE THE SPELLING

The CRQ workflow package is `scheduler`, not `scheduler`. The front-end feature folder is `scheduler`. Both spellings are load-bearing in their own repository; neither is going to be renamed. Grep for both.

`BaseService` — what every service inherits

Nearly every `@Service` extends `BaseService`, which is not an abstraction so much as a bag of injected collaborators. Extending it gives you, with no constructor wiring:

MEMBER	WHAT IT IS
<code>jdbcTemplateOne</code> / <code>Two</code>	The two data sources (Chapter 8).
<code>databaseUtils</code>	The procedure-calling helper. This is where the real work happens.
<code>webClient</code>	Reactive HTTP client for Remedy, Cygnet and the Python data-agent server.
<code>config</code>	<code>AppPropertiesConfig</code> — the external config file's values.
<code>commonService</code>	Small shared helpers.
<code>passwordEncoder</code>	A <code>BCryptPasswordEncoder</code> .
<code>LOGGER</code> plus four named loggers	Per-class logger, and the dedicated streams for cancel-CRQ, external plan submission, CRQ status push and reschedule.

Controllers occasionally extend it too (`AuthController` does) when they need a logger or a template directly. That is tolerated, not encouraged.

Controllers

Controllers are thin and follow a consistent shape: a class-level `@RequestMapping` naming the feature, method-level mappings naming the action, and a return type of `ResponseEntity<...>`. Two extra concerns show up on them:

- `@PreAuthorize` for authority-based gating. Authorities are strings of the form `MODULE_SUBMODULE_PERMISSION`, assembled by `UserPermissionService.getPermissionsByUserIdV1` from the `USER_ROLE_MAP → ROLE_PERMISSION_MAP → SUB_MODULE → MODULE` join. The JWT filter only performs that query when the handler actually carries the annotation, so unannotated endpoints cost nothing.
- `@Auditable` for the audit trail — see below.

Error handling

`GlobalExceptionHandler` maps exceptions onto HTTP status codes and a consistent JSON body. The body shape is the `ApiResponse` record: `{ status, message }`.

EXCEPTION	RESULT
<code>MethodArgumentNotValidException</code>	400 with the field errors.
<code>BusinessException</code>	A refused-but-valid operation. Carries its own status.
<code>PermissionDeniedException</code>	403.
<code>StageActionBlockedException</code>	A dedicated <code>StageActionErrorResponse</code> — used by the CRQ stage guards so the UI can show <i>why</i> a stage transition was refused (Chapter 15).
<code>DatabaseOperationException</code>	500. Thrown by <code>DatabaseUtils</code> for anything the database reported.
<code>ExcelUploadNotFoundException</code>	404 for an unknown async upload job id.
<code>DataAccessException</code> , <code>Exception</code>	Catch-alls, 500.

Security failures do not reach this handler — they are produced inside the filter chain. `SecurityConfig` installs an authentication entry point and an access-denied handler that write the *same* `ApiResponse` JSON, so a missing token, an invalid token and a forbidden action all look alike to the front end. That uniformity is what lets the front end have exactly one global 401/403 handler.

The audit trail

The `audit` package adds an audit row for state-changing user actions with a single annotation on the controller method:

```
@Auditable(module = AuditModule.USER_MANAGEMENT,
            submodule = AuditModule.SUB_USER,
            action = AuditAction.UPDATE,
            affectedUserParam = "request.userId",
            remark = "Updated user details")
@PutMapping("/v1/updateemp")
public ResponseEntity<ApiResponse> updateEmployee(...) { ... }
```

Design points worth copying when you add auditing to a new endpoint:

- The aspect advises `@AfterReturning`, so a method that threw records nothing.
- It also inspects the return value and skips the write for a non-2xx `ResponseEntity` or an `ApiResponse` whose status is Fail or Error. CHM has several endpoints that report refusal with HTTP 200; an audit trail claiming those succeeded would be worse than none.
- Module and action names are plain `String` constants, not enums, because the underlying columns are free-text and already contain values written by other clients.
- Login is audited explicitly rather than by annotation, because at that moment the security context is still empty and the aspect could not resolve an actor.
- Only super admins can read the log. That check is enforced in the back end independently of the two front-end guards.

10

The Database Contract

Stored procedures are the interface between Java and the data. Feature code does not write `SELECT`. This is the single most important convention in the codebase, and it comes with rules that are easy to break silently.

The rule

From the migration README, verbatim in spirit: *stored procedures are the interface. The Java layer calls `CALL <proc>(...)` through `DatabaseUtils`; feature code does not write `SELECT/INSERT/UPDATE` text. A new read path means a new procedure, not new SQL in a service.*

The handful of raw queries that do exist are infrastructure, not features — the session lookup in `TokenValidationService`, the authority join in `UserPermissionService`, the credential lookup in `AuthService`.

Calling a procedure

`DatabaseUtils` offers a method per result shape. Choosing the right one is most of the job:

METHOD	USE WHEN
<code>executeProcedureAndFetchObjects(V1)</code>	A single result set mapped to a DTO list via <code>BeanPropertyRowMapper</code> . The everyday read.
<code>executeProcedureGetDataWithError</code>	Same, but the procedure may report a failure as an <code>error_message</code> column. Throws <code>DatabaseOperationException</code> if it sees one.
<code>executeProcedureSingleResultWithError</code>	The above, returning the first row or <code>null</code> .
<code>fetchOrderedKeyValueFromProcedure</code>	Column order matters and the shape is dynamic — returns <code>LinkedHashMap</code> rows.
<code>executeProcedureAndProvideKeyValueWithHeadersFormat</code>	The same, packaged as <code>{ headers, data }</code> for a generic table UI.
<code>updateUsingProcedure</code>	A write whose only outcome you need is success or failure. See the warning below.
<code>executeProcedureForMessage</code> / <code>...V1</code>	A write whose procedure returns a status/message row. <code>V1</code> returns <code>ApiResponse</code> .
<code>executeProcedureAllResultSets</code> / <code>...LastResultSet</code>	A procedure that emits several result sets. Several CHM procedures do.

METHOD	USE WHEN
<code>extractMultiPagedResult</code>	A paged read: returns <code>ProcedurePageResult<T></code> carrying <code>totalCount</code> and <code>data</code> .

TRAP 1 — `UPDATEUSINGPROCEDURE` REPORTS SUCCESS FOR A ROLLBACK

If a procedure rolls back internally and reports the failure as a message row, `updateUsingProcedure` still answers “Success”. The user sees a green toast and nothing has changed. When a procedure can refuse the operation, call `executeProcedureForMessageV1` instead — it surfaces the message.

TRAP 2 — A READ PROCEDURE MUST NEVER RETURN `ERROR_MESSAGE`

`executeProcedureGetDataWithError` treats a column called `error_message` as a thrown exception, which the API layer answers as HTTP 500. Several procedures were written to report “nothing found” that way. The result is a 500 on an empty search, which freezes RTK Query on stale data and looks like the UI ignored your filter. **“Nothing matched” is a normal answer and must come back as zero rows.**

TRAP 3 — COLUMN ALIASES AND `BEANPROPERTYROWMAPPER`

Spring lowercases a column label and matches it against the underscore form of the DTO property. `underscoreName` breaks on a case change and *never before a digit*. So `categorizationTier1` must be aliased `categorization_tier1`, not `categorization_tier_1`. An alias that gets this wrong binds to nothing and yields a silent `null` — no error, no warning, just a blank column in the UI.

Paging

Paging is done **inside the procedure** with `COUNT(*) OVER ()`, so a page and its true total arrive in one round trip and no separate `*_Count` procedure can drift out of sync with the `WHERE` clause. Page size is clamped inside the procedure. On the Java side, `PaginationUtils` and `ProcedurePageResult<T>` carry the result and `PageResponseDto` shapes the response.

Migrations

Schema and procedure changes live in `airtelmanagement/db/migration/`, one file per change, named `YYYY-MM-DD_what-it-does.sql`. There are around 45 of them. The conventions are strict and worth following exactly:

- **The date is when the change was authored**, not when it was applied, and a file is never renamed afterwards.
- **Every file opens with a header comment** stating what already existed and is left alone, what the file adds, why, and how to roll it back. `2026-09-04_ui_actions_audit_log.sql` is the current reference for the shape.
- **Files are additive and re-runnable.** Procedures are `DROP PROCEDURE IF EXISTS` then `CREATE PROCEDURE`; index creation is guarded against `information_schema` because MySQL has no `CREATE INDEX IF NOT EXISTS`. Running a migration twice must be a no-op, not an error.

TRAP 4 — THE LIVE DATABASE AND THE REPOSITORY DRIFT

Procedures have been hand-edited on the server. It has happened more than once that the deployed procedure had extra parameters, missing columns or an entirely different body compared with the `.sql` file in git — in one case a deployed procedure had been silently truncated by eight columns.

Before you change any procedure, dump what is actually deployed and diff it against the file in

`db/migration/` :

```
SHOW CREATE PROCEDURE Get_CRQ_Workflow_Overview;  
SELECT ROUTINE_DEFINITION FROM information_schema.ROUTINES  
WHERE ROUTINE_NAME = 'Get_CRQ_Workflow_Overview';
```

When invoking the MySQL CLI on a procedure body, pass `--comments` — without it the client strips comments and you lose the header that explains the procedure.

Naming

Procedure names are not consistent, and you cannot infer behaviour from them. The clearest example, and one that has caused a production-shaped bug: `SP_GET_MOP_DETAILS_BY_CRQN` reads like a getter but **inserts** a MOP, a MOP version and a MOP file row, and refuses a second call. It must never be called from a GET handler. Read the body before you trust a name.

11

Front-End Architecture

Feature-sliced folders, one Redux store, one RTK Query API instance that every feature injects into, and a router whose every entry is permission-gated.

The shape of `src/`

```
src/
├─ main.tsx          Providers, AuthHydrator, GlobalLoader, ToastContainer
├─ App.tsx           BrowserRouter + theme + AppLayout + /login vs everything else
├─ app/
│   ├─ store.ts      configureStore; root reducer resets on logout
│   ├─ hooks.ts      typed useAppSelector / useAppDispatch
│   ├─ store/        auth.storage.ts, postLoginRedirect.ts
│   └─ config/       env.ts, roles.config.ts, filters.config.ts
├─ service/api.ts    THE RTK Query instance – one createApi for the whole app
├─ routes/           AppRoutes, PrivateRoute, PublicRoute, DefaultRedirect, LegacyRedirect
├─ rbac/             navRegistry, routeAccess, usePermission, PermissionGate, AccessDenied
├─ components/       layout/ (AppLayout, Header, SideBar), common/, ui/, loading/
├─ style/            MUI theme, colour mode
└─ features/         one folder per functional area – see below
```

Anatomy of a feature

Every folder under `features/` follows the same internal layout. Copy it exactly when you add one:

```
features/<feature>/
├─ api/              <feature>ApiSlice.ts – api.injectEndpoints({...})
├─ components/       presentational + container components
├─ pages/            route-level screens
├─ hooks/            feature-local hooks
├─ types/            TypeScript types for this feature only
├─ slices/           (optional) a Redux slice for genuinely local UI state
├─ utils/            pure helpers
└─ index.ts          barrel – what the rest of the app may import
```

Nineteen features exist today. Their relative sizes tell you where the product's weight is:

FEATURE	FILES	WHAT IT IS
<code>scheduler</code>	162	The CRQ workflow and everything under it. By far the largest.
<code>roster</code> , <code>rosterGenerator</code>	47, 46	Roster viewing and roster generation.
<code>userMe</code>	44	Self-service: personal roster, leave, notification manager.
<code>cabManager</code>	41	Change Advisory Board.
<code>dashboard</code>	37	The employee dashboard widgets.
<code>userManagement</code> , <code>teamManagement</code>	34, 34	Users, logins, audit log; teams and employees.
<code>crqAnalytics</code> , <code>crqJourney</code>	31, 30	Analytics dashboards; the CRQ pipeline canvas.
<code>settings</code> , <code>orgHierarchy</code>	28, 14	Global settings, RBAC admin, organisation tree.
<code>dataAgent</code>	25	Natural-language-to-SQL chat and analytics.
<code>inbox</code> , <code>rescheduleNotification</code>	13, 21	Notification inbox and reschedule approvals.
<code>auth</code> , <code>myDashboard</code> , <code>sftpManagement</code> , <code>globalSearch</code>	16, 6, 8, 6	Login, the merged dashboard shell, SFTP tooling, global CRQ search.

State: what goes where

CHM has very little client state, and that is deliberate. The store holds four slices plus the RTK Query cache:

```
store
├─ api          RTK Query cache – server data lives here, not in slices
├─ auth         token, user, isAuthenticated, hydrated
├─ roster       local UI state for the roster grid
├─ attributeUpdate local UI state for the attribute-update dialog
├─ planViewAndSetup local UI state for plan authoring
```

THE ROOT REDUCER RESETS ON LOGOUT

`store.ts` wraps `combineReducers` so that dispatching `auth/logout` sets the whole state tree to `undefined`. Every logout path — the header menu, the global 401 handler, the cross-tab storage event — therefore clears roster, dialog and plan state without each call site remembering to. The RTK Query cache is cleared separately by `api.util.resetApiState()` in the 401 handler. Together these guarantee the next user on a shared machine can never see the previous user's data.

The single API instance

`src/service/api.ts` creates *one* `createApi` with an empty `endpoints` object and a long `tagTypes` list. Every feature then calls `api.injectEndpoints(...)`. There are 37 such slices. The benefits are one cache, one middleware, one base query and cross-feature tag invalidation.

The base query does three things:

1. Attaches `Authorization: Bearer <token>` from the auth slice.
2. Sets `credentials: "include"` so the `HttpOnly` `jwt` cookie travels too — a second, XSS-resistant way to authenticate if the header is ever missing.
3. Wraps the result: on **401** it clears storage, dispatches `logout`, resets the cache and toasts “session expired”; on **403** it toasts the server's message but leaves the session intact.

THE AUTH-LIFECYCLE EXEMPTION

Sign-in, logout and session-terminate are excluded from that handler. A 401 from `/auth/v1/session/terminate` means “wrong password”, not “your session expired” — the caller does not have a session yet, which is why they are on that screen. Without the exemption, a failed login would tear down state and toast a confusing message.

Writing an endpoint

```
export const rosterApiSlice = api.injectEndpoints({
  endpoints: (builder) => ({
    getRosterView: builder.query<RosterViewResponse, RosterViewParams>({
      query: ({ domainId, subDomainId, startDate, endDate }) => ({
        url: "/monthlyrosterview",
        method: "GET",
        params: { domainId, subDomainId, startDate, endDate },
      }),
      providesTags: ["RosterView"],
    }),
    changeShift: builder.mutation<{ success: boolean; message: string }, ChangeShiftParams>({
      query: (body) => ({
        url: "/monthlyrosterview/changeshift", method: "POST", params: body,
      }),
      invalidatesTags: ["RosterView"],
    }),
  }),
});
```

Tag names are declared centrally in `api.ts`. Note that some carry historical typos (`"RosterVieW"`) — match the existing spelling rather than fixing it, or the invalidation silently stops working.

Routing

`App.tsx` mounts a `BrowserRouter` whose `basename` comes from `import.meta.env.BASE_URL` — itself derived from `VITE_APP_BASE_PATH` — so no deployment path is ever hard-coded. Two routes exist at the top level: `/login` wrapped in `PublicRoute`, and `/*` handed to `AppRoutes`.

Inside `AppRoutes`, every page is loaded with `React.lazy` and every route group is wrapped in `PrivateRoute`. Four guard components share the work:

COMPONENT	RESPONSIBILITY
<code>PrivateRoute</code>	Waits for hydration; redirects to <code>/login</code> when unauthenticated; then asks <code>isPathAllowed()</code> whether this exact pathname is permitted and renders <code>AccessDenied</code> if not.
<code>PublicRoute</code>	Wraps <code>/login</code> . If an already-authenticated user lands there, sends them to their first accessible path.
<code>DefaultRedirect</code>	The index and catch-all route. Computes the user's first accessible path; renders <code>AccessDenied reason="no-modules"</code> when there is none. There is no hard-coded home page.
<code>LegacyRedirect</code>	Keeps retired URLs working, carrying query string, hash and location state across.

DIRECT-URL ACCESS IS GUARDED, NOT JUST HIDDEN

`PrivateRoute` re-evaluates the full pathname on every navigation. Hiding a sidebar item is never the security boundary — typing the URL hits the same predicate, and the back end refuses the data independently of both.

Loading, and why there is a registry for it

`LoadingProvider` exposes `useGlobalLoading(active, key)`. Only the real bootstrap window — token validation on first paint — drives the full-screen loader; an earlier design that showed it for any in-flight query made the app flash constantly. Route-level suspense uses `RouteFallback`; sections use `SectionOverlayLoader` and skeletons.

Theme and layout

MUI 7 with Emotion. `ColorModeContext` and `useMode()` in `style/theme` provide light and dark modes; dark mode is built on one exported five-step neutral ladder so surfaces stay consistent. `AppLayout` renders the header and sidebar (`showChrome` is false until authenticated and hydrated, so the login page is chrome-free). The sidebar's contents come from `useSidebarNav()`, which filters the nav registry through the user's permissions.

Code splitting and bundling

- Every route element is `React.lazy`. Named exports are pulled through a feature barrel so the bundler still emits one chunk per feature.
- `manualChunks` collapses `node_modules` into a single `vendor` chunk. Splitting React, MUI and Emotion apart causes real runtime failures — “Cannot access X before initialization” — because of circular internal references across chunk boundaries.
- `pdfjs-dist` and `react-pdf` are the one deliberate split: ~430KB needed only by the MOP validation workspace. The regex is anchored on the package directory so it cannot accidentally catch `@react-pdf/renderer`, a different library used by the team-management export.

- Because everything in `node_modules` lands in `vendor`, wrapping a heavy third-party library in `React.lazy` alone defers nothing — it must also be given its own `manualChunks` branch.

12

Authentication and Sessions

A JWT proves who you are; a database row proves your session is still live. Both must agree on every single request. Most login-related bugs in CHM's history come from forgetting the second half.

The two halves of a session

WHAT MUST BE TRUE FOR A REQUEST TO SUCCEED

The JWT

Signed with `jwt.secret`. Subject = numeric user id.
`jti` = a UUID minted at login. Expiry seven days.

The session row

`AUTH_JWT_TOKENS` where `token_id` = that
`jti`, `valid = true` and `expires_at > NOW()`.

both checked on every request by `JwtAuthenticationFilter`

If either fails the response is `401 { "status": "Fail", "message": "Invalid session" }` and the front end signs the user out.

Login

1. `AuthService.findPasswordByOlmId` reads `AUTH_CREDENTIAL` for the username and verifies the password (or the configured master password).
2. It checks for a live session with the shared `LIVE_SESSION_PREDICATE` (`valid = true AND expires_at > NOW()`) and returns `Already Logged` if one exists.
3. Otherwise it deletes any previous row for the user and inserts a new one with a fresh `token_id`. Because the id is new every time, matching on it alone already accounts for logout and for re-login elsewhere.
4. `AuthController` mints the JWT, sets the `HttpOnly` cookie, writes an explicit `LOGIN` audit row and returns `accessToken`.

The filter

`JwtAuthenticationFilter` runs before Spring Security's authorization check. Four behaviours deserve attention:

- **It skips public endpoints entirely.** `permitAll()` does *not* stop a filter added with `addFilterBefore` from running, so a stale token left in a browser used to make even *login itself* fail with “Invalid session”.

The fix was to share one list, `PublicEndpoints.PATTERNS`, between the filter's `shouldNotFilter` and `SecurityConfig`'s matchers. Two hand-maintained copies had drifted.

- `/auth/v1/logout` is **deliberately not public**. Ending a session requires proving whose it is. The `oIdmId` in the request body is accepted and ignored; invalidation keys off the `token_id` in the caller's own verified token.
- **It reads the token from the header or the cookie.** Either is accepted.
- **Authorities are loaded lazily.** The permission query only runs when the matched handler carries `@PreAuthorize`.

SIMPLE ACCESS TOKENS

`/usermanagement/v1/getaccesstoken` issues a different kind of token whose subject is a raw username rather than a numeric id, with no session row behind it. These are a static integration credential. The filter recognises them (the subject does not match `\d+`), validates them separately, and **refuses them on any handler carrying `@PreAuthorize`** — they can never gain an authority.

Public endpoints

The complete list, from `PublicEndpoints.PATTERNS`:

`/test/**` `/auth/v1/signin` `/auth/v1/session/terminate` `/actuator/**` `/users/signup` `/api/python/runScript`
`/CHMDataAdapter/CreateCrq` `/usermanagement/v1/getaccesstoken`

Adding an endpoint here makes it reachable from anywhere the service is. Do it only for machine-to-machine integrations, and put the list entry and the security reasoning in the same commit.

Front-end bootstrap: `AuthHydrator`

Mounted once in `main.tsx`, above the router. On a cold start it:

1. Reads the token from `localStorage`. If there is none, it records the currently requested in-app path as a pending post-login redirect (*the only place that is ever written*), dispatches `logout` and finishes hydration.
2. If there is one, it paints the cached user immediately so the UI does not blank out, then calls `/users/v2/getloggeduserdetails` to **re-validate the token against the back end** and refresh the permission tree. On failure it clears storage and logs out.
3. Registers a `storage` listener for cross-tab sign-in and sign-out, and a `pageshow` listener so a bfcache restore (pressing Back after logout) re-validates instead of trusting the painted snapshot.

WHY STORAGE MOVED FROM SESSION TO LOCAL

The session used to live in `sessionStorage`, so a refresh or a new tab logged the user out. It is now `localStorage`, shared across tabs. That is only safe because the token is re-validated against the back end on every bootstrap — a cleared or expired session row cannot be skipped by a stale local copy.

Post-login redirect

`postLoginRedirect` is deliberately `sessionStorage`-scoped: it describes *this tab's* interrupted navigation and can never bleed into another. It is written in exactly one place, consumed once, and cleared by every session teardown — so a signed-out user's last page can never be handed to whoever signs in next on a shared machine. The value is also vetted through `isPathAllowed()` before being honoured.

Logout paths

TRIGGER	WHAT HAPPENS
Header menu	<code>POST /auth/v1/logout</code> invalidates the session row by <code>token_id</code> ; the client clears storage and dispatches <code>logout</code> .
Any 401 response	The RTK Query base query clears storage, dispatches <code>logout</code> , resets the query cache and toasts.
Another tab signing out	The <code>storage</code> event fires in every other tab, which clear and log out.
Failed re-validation at bootstrap	Same as above, silently.

13

Authorization: the RBAC Model

What a user can see is decided by data, not by code. There is exactly one file that maps URLs to required grants, and every guard in the app reads it.

The model

```
Role → Module → Sub-module → Permission
                                VIEW · CREATE · UPDATE · DELETE · APPROVE · REJECT
```

A role is granted permissions on sub-modules. A module counts as granted when any of its sub-modules carries at least one permission. The grants live in the `WEB_*` tables and are edited at runtime through **Global Settings › Admin Settings**.

SUB-MODULE NAMES ARE DATA, AND THEY CONTAIN TYPOS

Gating is by exact name match against live rows. Under the Scheduler module the row really is spelled `"Plan View And Setup"`, with that capital I. Matching it exactly is what makes the grant work; “fixing” the spelling in the registry silently removes the page from everyone. Likewise the module name is `"Roster Management"`. Verify against the database, never against your intuition.

What the front end receives

`/users/v2/getloggeduserdetails` returns the user plus the full hierarchy:

```
{
  "olmId": "...", "employeeName": "...", "roleCode": "TEAM_LEAD", "userId": "...",
  "modules": [
    { "moduleId": 5, "moduleCode": "...", "moduleName": "Scheduler",
      "subModules": [
        { "subModuleId": 12, "subModuleName": "Shift Scheduler",
          "permissions": [ { "permissionCode": "VIEW", ... } ] }
      ] }
  ]
}
```

`rbacNormalizer` turns this into two shapes held in the auth slice: a flat `Record<moduleName, permissionCode[]>` for cheap module-level checks, and the untouched hierarchy for sub-module checks.

The predicates

`rbac/permissionCore.ts` holds pure functions with no Redux dependency, so they can be evaluated against a user object that has not been committed to the store yet — which is exactly what the login page

needs when it computes where to send you. `usePermission()` wraps them for components:

API	MEANING
<code>can(module, action)</code>	Does the user hold this action on this module?
<code>hasModule(module)</code>	Any access at all to the module.
<code>hasSubModule(module, sub)</code>	Any access to a specific sub-module.
<code>canAll</code> / <code>canAny</code>	Several actions at once.
<code>isSuperAdmin</code> , <code>roleCode</code>	The raw role, and the one hard-coded role predicate in the app.

THERE ARE TWO `USEPERMISSION` HOOKS

The canonical one is `src/rbac/usePermission.ts`. A second, older hook lives at `src/features/auth/hooks/usePermission.ts`. Import the `rbac/` one in new code; check which one an existing screen uses before you conclude its gating is wrong.

The nav registry — one declaration, four consumers

`src/rbac/navRegistry.tsx` declares every top-level item and its children:

```
{
  to: "/scheduler",
  text: "Scheduler",
  icon: <ScheduleIcon />,
  requiredModule: "Scheduler",
  children: [
    { to: "/scheduler/crqWorkflow", text: "CRQ Workflow",
      requiredModule: "Scheduler", requiredSubModule: "Shift Scheduler",
      matchPaths: ["/scheduler/crqWorkflow"] },
    ...
  ],
}
```

Four things read that one declaration:

1. `useSidebarNav()` — what appears in the sidebar.
2. `routeAccess.isPathAllowed()` — whether a typed URL is permitted; used by `PrivateRoute`.
3. `getFirstAccessiblePath()` — the per-user default landing page.
4. In-page tab strips, which gate their tabs with the same predicate.

Because the same predicate serves all four, the sidebar and the route guard cannot disagree about a URL. `routeAccess` flattens parents and children, sorts by longest path first so a child wins over its parent, and treats an unregistered path as allowed — a safety net for a route added without a nav entry, rather than silently locking everyone out.

Four grant shapes

FIELD	EFFECT
<code>requiredModule</code>	Module-level gate. <code>null</code> means open to any signed-in user.
<code>requiredSubModule</code>	Tightens to one sub-module of that module.
<code>requiredAnyOf</code>	Allowed if any listed requirement is satisfied. My Dashboard uses this because it spans the “Dashboard” and “Me” modules.
<code>requiredSuperAdmin</code>	Only a super admin, whatever modules they hold. Used by the Audit Log so that access cannot be delegated from Global Settings.

Route-only entries

`ROUTE_ONLY_ACCESS_ENTRIES` covers screens that are guarded but not shown in the sidebar — today, `/user-management/auditlog`. Without it, that path would prefix-match the plain `/user-management` entry and open to anyone holding the User Management module.

Two deliberate module-level fallbacks

`MODULE_ONLY_GUARD` downgrades sub-module enforcement to module level for *Cab Manager* and *SFTP Management*. In both cases the live sub-module rows do not match the names in the registry — CAB falls back to a persona table when a user has no sub-module rows, and SFTP’s live rows are named Dashboard/Files/Servers rather than Windows/Linux. Enforcing them at the route layer would lock real users out of every page in those modules.

A GATING BUG WORTH LEARNING FROM

The Scheduler children once declared `requiredModule: "Role-Based Access Control"` — the admin RBAC module, copy-pasted rather than intended. The effect was that a Scheduler page only appeared for someone holding *both* Scheduler and admin RBAC, so every role granted Scheduler alone saw the parent item with no children and could never reach CRQ Workflow. The identical bug hit Inbox, which was gated on “Notification System” (the admin-side config module) instead of “Inbox”. When a page is invisible for a role that should have it, suspect the module *name* first.

Enforcement is three-deep

1. **Sidebar** — the item is not rendered. Convenience, not security.
2. **Route guard** — `PrivateRoute` refuses the pathname and renders `AccessDenied`.
3. **Back end** — `@PreAuthorize`, or an explicit check inside the service, refuses the data. The audit log additionally has a procedure, `sp_get_ui_actions_log_access`, that answers the same question server-side.

A change to visibility that only touches layer one is not a security change. Say so in your PR and make sure layer three exists.

Organisation scoping is separate from RBAC

Permissions decide *which screens*; the organisation hierarchy decides *which rows*. The Vertical / Function / Domain / Sub-domain pickers at the top of most screens are derived from the user's Organization Hierarchy grants, with a legacy visibility table as fallback. Two similarly named things are not the same taxonomy: an organisation `domain_id` and the `chm_domain` on a plan are different trees. Confusing them yields empty screens with no error.

PART FOUR

The Functional Modules

A reference, not a narrative. Chapter 14 is the index — the table that connects a sidebar item to a front-end folder, a URL, a back-end package and an API base path. The chapters after it go one level deeper on each area, with the most detail on the CRQ workflow, which is where most work lands.

14	Module map — the index
15	CRQ Workflow — the seven stages
16	CAB Manager
17	Roster — view and generation
18	People — team and user management
19	Configuration — global settings and org hierarchy
20	Inbox and notifications
21	Analytics, Data Agent, SFTP and Audit

14

Module Map

One table connecting what a user clicks to the code you will edit. Keep a finger here while you read the rest of Part IV.

SIDEBAR ITEM (RBAC MODULE)	ROUTE	FE FOLDER	BE PACKAGE · API BASE
My Dashboard Dashboard, Me	/my-dashboard/... overview, monthly-view, leave, notifications	myDashboard , dashboard , userMe	dashboard , me , attendance , rosterview /dashboard , /leave , /attendance , /notification- manager
Cab Manager	/cabmanager/...	cabManager	cabmanager /cab/crqs , /cab/sessions , /cab/planning , /cab/dashboard , /cab/admin
Team Management	/team/teammanagement	teamManagement	teammanagement /teamoverview , /teamoverview/excel
Scheduler	/scheduler/crqWorkflow /scheduler/planviewandsetup /scheduler/taskconfig /scheduler/crqjourney /scheduler/cancelledcrq	scheduler , crqJourney	scheduler , activity , impactbatch , remedy , sygnet /crqworkflow , /crq/reschedule , /plan , /activity , /scheduler
Roster "Roster Managemement"	/roster/view /roster/generation	roster , rosterGenerator	rosterview , rostergeneration /monthlyrosterview , /rostergenration , /goldenset
Inbox	/inbox/notifications	inbox , rescheduleNotification	notification /notification
User Management	/user-management/usermang /user-management/userlogs /user-management/auditlog (super admin)	userManagement	usermanagement , user , audit /usermanagement , /users , /audit-logs
Analytics CRQ Analytics	/analytics/dashboard /analytics/crq-analytics /analytics/reports	crqAnalytics	crqanalytic , analyticsreport /crq-analytics-new , /analytics/reports

SIDEBAR ITEM (RBAC MODULE)	ROUTE	FE FOLDER	BE PACKAGE · API BASE
SFTP Management	/sftp-management/windows /sftp-management/linux	sftpManagement	filetransfer /api/sftpfromui , /api/sftp/linux/remote
Global Settings	/global- settings/networkfreezsetting /global- settings/adminsetting /global-settings/orgconfig	settings , orgHierarchy	globalsettings , orghierarchy /networkfreez , /global- settings/permissions , /global- settings/org-hierarchy
(not in sidebar) Data Agent	/data-agent	dataAgent	dataagent /dataagent

Module names in the second column are the exact strings the nav registry matches against live **WEB_MODULE** rows — including the two known misspellings.

Back-end packages with no sidebar item

Several packages exist purely to serve integrations or other modules:

PACKAGE	PURPOSE
remedy	Outbound calls to Remedy/Helix — CAB requests, planning-in-progress, SPOC updates, plan submission.
sygnet	Slot booking, calendar view and CRQ status push (/api/slots , /api/calendar , /crq_status).
remedycygnetsync	The periodic reconciliation job and its manual trigger endpoints.
impactbatch	Impact-analysis batch CSV listing and Excel generation.
attributeupdate	The CRQ attribute-update dialog's dropdowns and save.
attendance	Clock in/out and work mode for the dashboard.
audit	The audit aspect, writer and read API.

15

CRQ Workflow: the Seven Stages

The heart of the product and the largest feature in either repository. Everything here is organised around one idea: a CRQ occupies exactly one stage at a time, and that fact lives in

`CRQ_MASTER_TBL`.

The stages

1	2	3	4	5	6	7
Plan & Inventory	Impact Analysis	MOP Create	MOP Validate	Scheduling	Activity Implement	Closure
VALIDATE	IMPACT_ANALYSIS	MOP_CREATION	MOP_VALIDATION	SCHEDULING_APPROVAL	EXECUTION	CLOSURE

The database enum values are on the second line of each box. The front end uses shorter keys — `review`, `impactanalysis`, `mopcreate`, `mopvalidate`, `scheduling`, `activityimplement`, `closer` — and both directions of that mapping are declared twice, once in `CrqWorkflowService.STAGE_KEYS` (back end) and once in `constants/workflowStages.ts` (front end). If you add a stage you must edit both.

The data model

TABLE	ROLE
<code>CRQ_MASTER_TBL</code>	Authoritative. <code>current_stage</code> plus <code>current_status</code> is the truth about where a CRQ is.
<code>CRQ_STAGE_ASSIGN_TBL</code>	Per-stage assignment and timing rows.
<code>CRQ_HISTORY_TBL</code>	The audit trail of stage transitions.
<code>CRQ_PHASES_STATUS_TBL</code>	Legacy. No update procedure writes to it any more; historic rows were backfilled into the model above. Do not read it in new code.
<code>CRQ_VALIDATION_DETAILS_TBL</code>	Backs the Plan & Inventory Validate dialog.
<code>CRQ_CANCEL_TBL</code>	Cancellation detail — but note that the <i>fact</i> of cancellation is <code>CRQ_MASTER_TBL.current_status = 'CANCELLED'</code> , not the presence of a row here.
<code>CRQ_DETAIL_TBL</code>	Present but empty in the environments checked. Do not build on it without confirming.

`Get_CRQ_Stage_History` returns, in one call, every stage row for every CRQ in a domain and sub-domain, so a listing can attach history without an N+1. Its status derivation is worth reading once: for the current stage it maps `current_status` onto a display word; for stages before it in `FIELD()` order it reports *Done*; for stages after it, *Not Started*.

The stage-driven front end

Six of the seven stages share one implementation. `constants/stageConfig.ts` holds `STAGE_CONFIG_MAP`, a single source of truth keyed by stage:

```
impactanalysis: buildStageConfig({
  key: "impactanalysis",
  label: "Impact Analysis",
  // endpointBase → /crqworkflow/updateimpactanalysis/{start|pause|done}
  // reviewQueryUrl → the GET that lists the CRQs sitting at this stage
  endpointBase: "updateimpactanalysis",
  reviewQueryUrl: "/crqworkflow/impactanalysis",
  stageEnum: "IMPACT_ANALYSIS",
  statusField: "impactAnalysisStatus",
  olmIdField: "olmidImpactAnalysis",
  startDateField: "impactAnalysisStartDate",
  endDateField: "impactAnalysisEndDate",
  buildDonePayload: (values, crq, ctx) =>
    buildCommonDonePayload(values, crq, ctx, { taskNumber: taskNumbersOf(crq) }),
})
```

The table page, the action dialog, the card, the hooks and the API slice are all driven off that map. Adding a stage is:

1. Add its key to `StageKey` in `types/stageWorkflow.types.ts`.
2. Add one entry to `STAGE_CONFIG_MAP`.
3. Optionally override `fields` or `statusOptions` if its form differs.

Plan & Inventory is the exception. It predates the generic map and runs on its own endpoints (`crqreviewApiSlice`, `schedulerApiSlice`), which is why its stage id is `"review"` and it is not a `StageKey`.

Every stage's action shape

Three verbs per stage: `/start`, `/pause`, `/done`. The outcome options are shared — *Pass* (`Done`), *Failed*, and *Cancelled* — and choosing Cancelled reveals a shared cancellation block: which team to send the activity back to, the Remedy status, a reason, a derived rollback owner and a Remedy remark. That block is declared once in `constants/sharedFields.ts` and spread into every stage.

REFUSALS ARE FIRST-CLASS

`Update_CRQ_Scheduling_To_Done_Or_Failed` has five branches that refuse the transition. Those are surfaced as proper 409/404 responses via `StageActionBlockedException` and rendered as an inline alert in the dialog, rather than a generic failure toast. When you add a guard to a stage procedure, wire it the same way — the user needs to know *which* precondition failed.

Screens under Scheduler

SCREEN	WHAT IT DOES
CRQ Workflow <code>/scheduler/crqWorkflow</code>	The cockpit. A paged, filterable list of CRQs; drilling in gives the stage rail, the stage summary grid, the action panel and the history table. Endpoints: <code>/crqworkflow/overview/paged</code> , <code>/crqworkflow/overview/{crqNo}</code> .

SCREEN	WHAT IT DOES
Global search	Searches CRQs across the whole estate rather than the current org scope, via <code>Get_CRQ_Global_Search</code> and <code>GET /crqworkflow/search</code> . Note that <code>/overview/{crqNo}</code> hard-filters by domain and cannot go global, which is why a separate procedure exists.
Plan View & Setup <code>/scheduler/planviewandsetup</code>	Authoring plans and activities. Includes an Excel bulk upload whose procedure takes 37 name-based parameters — verticals, functions, domains, sub-domains and teams are resolved by name inside the procedure, not by id from the client.
Task Config <code>/scheduler/taskconfig</code>	Task-level configuration under a plan.
CRQ Journey <code>/scheduler/crqjourney</code>	The pipeline canvas. <i>Shared with</i> <code>/cabmanager/journey</code> — the same components render both, so an edit here changes both screens.
Cancelled CRQs <code>/scheduler/cancelledcrq</code>	A read-only register of every cancelled CRQ across all seven stages.

CRQ JOURNEY: THREE RESULT SETS, AND A HANDS-OFF PROCEDURE

`sp_get_crq_journey_page` emits three result sets; only the first was originally consumed, so pending-approval data was being discarded. The procedure itself is out of bounds for changes — enrichment lives in front-end utilities instead. Its row order and column names have also drifted twice, so **resolve rows by name, never by index**.

Reschedule

A five-step wizard under `/crq/reschedule`: context → reason → a calendar of candidate dates → slots for the chosen date → confirmation. Cancelling is a first class action (`CRQ_SP_RESCHEDULE_CANCEL`), and the reason is a dropdown backed by `/crq/reschedule/reason-options` plus free text. An earlier three-reschedule cap was removed.

Impact analysis and checkpoints

- **Batches.** Reading impact data is strictly two steps: `GET /impact/statuscsv/batch` lists the available batches and their modified dates, then `GET /crqworkflow/impactanalysis/batch` is called with that exact date. Do not guess a date.
- **Scripts.** `POST /crqworkflow/impactanalysis-script` and `POST /crqworkflow/refetchcheckpointscrip` run remote Python over SSH; the paths come from the external config file.
- **Checkpoints.** The CRQ Review screen shows an SFTP-backed checkpoint summary with a refresh action.

MOP create and validate

MOP Create collects the document metadata and accepts a PDF upload (`POST /crqworkflow/mopcreate/{crqNo}/pdf` , multipart). MOP Validate opens a workspace: the PDF rendered

with `react-pdf` alongside a findings list, where a reviewer raises findings, changes their state, and finally validates or rejects.

A PROCEDURE NAMED GET THAT CREATES

`SP_GET_MOP_DETAILS_BY_CRQN` inserts a MOP, a MOP version and a MOP file row, and refuses a second call. It must never be invoked from a GET handler, and the procedure itself must not be changed. This is the canonical example of why you read a procedure body before trusting its name.

16

CAB Manager

The approval side of change management. Its defining quirk: approvals are keyed by *service*, not by CRQ, because one change affects several services and each has its own approver.

Screens

SCREEN	ROUTE	PURPOSE
Cab Sessions	<code>/cabmanager/sessions</code>	Scheduled CAB meetings. The index route redirects here, because every persona has it.
Cab Planning	<code>/cabmanager/planning</code>	The queue of changes awaiting a CAB date, and the available dates.
My CRQs	<code>/cabmanager/mycrqs</code>	Changes awaiting <i>this</i> approver's decision.
All CRQs	<code>/cabmanager/allcrqs</code>	Everything, with a detail drawer, conflict check and a per-service CSV export.
CRQ Journey	<code>/cabmanager/journey</code>	The shared pipeline canvas (same components as the Scheduler route).
Implementation	<code>/cabmanager/implementation</code>	Execution tracking, ring by ring — proceed or block.
Admin Config	<code>/cabmanager/admin</code>	Six tabs: analytics, assignment matrix, escalation matrix, rejection reasons, users, audit.
Dashboard	<code>/cabmanager/dashboard</code>	Hidden since August 2026. The route stays mounted so old links resolve.

The decision actions

All on `/cab/crqs` and all keyed by `serviceApprovalId`:

POST `/cab/crqs/{id}/approve`

POST `/cab/crqs/{id}/reject`

POST `/cab/crqs/{id}/delegate`

POST `/cab/crqs/{id}/reschedule`

POST `/cab/crqs/reschedule-requests/{crqNo}/approve`

POST `/cab/crqs/{crqId}/assign-spoc`

POST `/cab/crqs/{crqId}/assign-fe`

Rejection reasons are themselves managed data — full CRUD at `/cab/crqs/cabrejectreasons` — which is why the Inbox reject dialog for a CAB item shows a picklist rather than a free-text box.

Conflict check

Both CRQ drawers carry a *Conflict* button backed by `GET /cab/crqs/{crqNo}/conflicts` and a decision endpoint. It answers whether the proposed window collides with another change. The backing procedure

existed but was unused and broken (an undeclared variable) until it was fixed live — a reminder that an unused procedure in this codebase is not a tested procedure.

Screen visibility — the one module that is different

`resolveCabScreens()` in `features/cabManager/rbac/cabScreens.ts` decides which CAB screens a user sees, and it is the only place in the app with a fallback:

1. Prefer `WEB_SUB_MODULE` grants from the login payload — the mechanism every other module uses, so assigning “All CRQs” to a role in Global Settings shows the page.
2. If the user's Cab Manager module carries *no* recognised sub-module rows — older deployments where the module was granted without a hierarchy — fall back to a hard-coded `ROLE_SCREEN` persona table so those users keep the tabs they had.

This fallback is why `MODULE_ONLY_GUARD` (Chapter 13) downgrades the route guard for Cab Manager to module level: the route layer has no equivalent fallback and would lock those users out of every CAB page.

One resolver serves both the sidebar children and the in-page tab strip, so the two can never disagree about a screen.

17

Roster: View and Generation

Two related features with very different characters. Viewing is a read-mostly grid with a few write actions; generation is an interactive planning tool built around a “golden set” baseline.

Roster View

`/roster/view`, feature folder `roster`, back end `rosterview` under `/monthlyrosterview`.

ENDPOINT	USE
GET <code>/monthlyrosterview</code>	The grid, scoped by <code>domainId</code> , <code>subDomainId</code> and a date range.
GET <code>/monthlyrosterview/userroster</code>	Self-scoped — the signed-in user's own roster, no org filters. Backs the My Roster card on the dashboard.
GET <code>/monthlyrosterview/currentshiftcount</code>	Headline counts per shift.
GET <code>/monthlyrosterview/shiftdropdowns</code>	Shift options for the change dialog.
POST <code>/monthlyrosterview/changeshift</code>	Manager reassigns a shift.
POST <code>/monthlyrosterview/shiftswapbymanager</code>	Manager-initiated swap.
POST <code>/monthlyrosterview/shiftswapreqbyteamember</code>	Engineer requests a swap — raises an inbox item needing two decisions.

The small *i* button on a roster cell opens a dialog showing that engineer's daily assignments and a Gantt-style timeline, backed by `sp_engineer_daily_assignments`.

Roster Generation

`/roster/generation`, feature folder `rosterGenerator`, back end `rostergeneration`.

The screen is built around the **Golden Grid** — an editable matrix of employees against shift days. It is the most interaction-heavy component in the product: bulk-action bar, brush-based painting of shifts, per-week override popovers, keyboard shortcuts, a filter drawer, level badges and an analytics modal. The logic is split across focused hooks (`useGoldenGridData`, `useGoldenGridEditing`, `useGoldenGridFilters`, `useGoldenGridKeyboardShortcuts`, `useGoldenGridTokens`) rather than one large component — follow that pattern if you extend it.

ENDPOINT	USE
GET <code>/goldenset</code> , POST <code>/goldenset/dailygoldenset</code>	Read and write the baseline pattern.
GET <code>/rostergeneration/futureweek</code>	The forward weeks being generated. Note the spelling of the path.
POST <code>/rostergeneration/futureweek/update</code>	Commit changes to a future week.

THE BLANK-SHIFT KEY BUG — A JACKSON NAMING TRAP

Roster grid keys are of the form `w1D1` (week 1, day 1). Jackson's default property naming mangles such a key on serialisation: `w1D1` goes out as `w1d1`, and the front end's lookup misses, rendering a blank cell. If a grid cell is inexplicably empty, compare the JSON key on the wire against the key the component asks for before looking anywhere else.

Holidays and network freeze

Roster generation and CRQ scheduling both respect the holiday calendar and network-freeze windows, which are configured in Global Settings under `/networkfreez` (Chapter 19).

18

People: Team and User Management

Three distinct surfaces deal with people, and they are easy to confuse. Know which one you are in before you change anything.

The three surfaces

SURFACE	ROUTE	WHAT IT OWNS
Team Management	<code>/team/teammanagement</code>	Employees within a sub-domain: create, edit, activate/deactivate, bulk upload. Back end <code>/teamoverview</code> .
User Management	<code>/user-management/usermang</code>	The administrative user list and profile view. Reuses the Team Overview create/update/status endpoints rather than duplicating them.
My Dashboard	<code>/my-dashboard/...</code>	The signed-in user's own view: attendance, roster, leave, notification preferences.

Team Management endpoints

ENDPOINT	USE
GET <code>/teamoverview/getusers</code>	The list.
GET <code>/teamoverview/getuserprofile/{userId}</code>	One profile.
GET <code>/teamoverview/getcreateuserdropdowns</code>	Every dropdown the create form needs, in one call.
POST <code>/teamoverview/v2/addnewemp</code>	Create. (<code>PUT /v1/addnewemp</code> is the older shape, still present.)
PUT <code>/teamoverview/v1/updateemp</code>	Edit.
PUT <code>/teamoverview/v1/updateuserstatus</code>	Activate / deactivate.
GET <code>/teamoverview/getempcountbysubdomainid</code>	Headline counts.

A SILENT NO-OP WORTH REMEMBERING

Edit Team Member once appeared to succeed while changing nothing: `sp_update_user` read a `role_id` column that does not exist on `USER_MASTER`, and an empty `SQL EXCEPTION` handler swallowed the error and reported Success. **Never write a bare `DECLARE EXIT HANDLER FOR SQL EXCEPTION` that returns a success row.** If a procedure can fail, it must say so, and the caller must use `executeProcedureForMessageV1`.

Bulk employee upload

Under `/teamoverview/excel`, and structured as `validate → batch → async`:

1. `GET /v1/template` hands the user the spreadsheet to fill in.
2. `POST /v1/upload` parses and validates without committing.
3. `POST /v1/batch` or `POST /v1/upload/async` commits; the async form returns an upload id.
4. `GET /v1/upload/{uploadId}/status`, `.../result` and `.../error-report` are polled by the UI and produce a downloadable error report.

The job store is in memory, so an upload in flight does not survive a restart. The underlying procedure, `sp_create_user_excel_sheet`, was left untouched by this redesign.

User logs and the audit log

- `/user-management/userlogs` — login history, from `GET /usermanagement/logindetails`.
- `/user-management/auditlog` — the UI actions log. **Super admin only**, guarded three times over (Chapter 13): a route-only registry entry, a fail-closed check in the page itself, and the back end's own check. It reads `GET /audit-logs` with `GET /audit-logs/filters` supplying the filter options.

19

Configuration: Global Settings and Org Hierarchy

Three tabs that change the product for everyone else. Treat changes here the way you would treat a schema migration.

Network Settings

`/global-settings/networkfreezsetting`, back end `/networkfreez`. Two entities, each with full CRUD:

- **Holidays** — `/holiday/view`, `/holiday/insert`, `/holiday/update`, `/holiday/delete/{holidayId}`, plus a location dropdown.
- **Network freeze days** — `/networkfreezedays/view`, `/networkfreeze/insert`, `/update`, `/delete/{freezeId}`.

Both feed roster generation and CRQ scheduling: a freeze window makes a slot unavailable.

Admin Settings — the RBAC editor

`/global-settings/adminsetting`, back end `/global-settings/permissions`. This is the largest controller in the codebase by endpoint count, and it is worth scanning once to understand the RBAC lifecycle:

GROUP	ENDPOINTS
Dropdowns	<code>/dropdown/roles</code> , <code>/modules</code> , <code>/modules-for-role</code> , <code>/unassigned-modules</code> , <code>/module-by-role-code</code> , <code>/sub-modules</code> , <code>/permissions</code>
Grants	<code>PATCH /update</code> , <code>PATCH /bulk-update</code> , <code>DELETE /reset/role/{roleId}/sub-module/{subModuleId}</code>
Roles	<code>/create-new-role</code> , <code>/rename-role</code> , <code>/disable-role</code> , <code>/add-role-permission</code>
Modules	<code>/create-new-module</code> , <code>/rename-module</code> , <code>/disable-module</code> , <code>/assign-module-to-role</code> , <code>/delete-module-for-role</code>
Sub-modules & permissions	<code>/create-new-sub-module</code> , <code>/rename-sub-module</code> , <code>/delete-sub-module</code> , <code>/create-new-permission</code>
Read-back	<code>/role-permissions</code> , <code>/role-permission-map</code>

RENAMING IS A BREAKING CHANGE

The nav registry gates on module and sub-module *names*. Renaming “Scheduler” or “Plan View And Setup” through this screen removes the corresponding pages from every user until the registry is updated to match. If you must rename, ship the registry change in the same release.

Organization Settings

`/global-settings/orgconfig` , back end `/global-settings/org-hierarchy` . CRUD plus a status toggle for each of the four levels — `/verticals` , `/functions` , `/domains` , `/sub-domains` .

These grants do more than shape a settings screen: the Vertical / Function / Domain / Sub-domain pickers at the top of most screens across the app are derived from a user's Organization Hierarchy grants (module 20), with a legacy visibility table as fallback. Those grants were once inert app-wide, which is why some older code still consults the legacy table.

A safety note on the procedures behind this module

The RBAC administration procedures live only in the live MySQL instance for several historically added features — they are not all represented in `db/migration/` . Dump before you edit, and if you author a fix, add the migration file so the next person does not face the same gap.

20

Inbox and Notifications

One list, one detail pane, and a small registry that decides which decision buttons appear.

There is no notification “type” column — the type is derived from two flags.

How a notification's type is decided

```
isActionable = 1, status = "PENDING"           → APPROVAL_REQUIRED  "Action Required"
isActionable = 1, status = COMPLETED / CLOSED → APPROVAL_COMPLETED  "Approval Completed"
isActionable = 0                               → INFORMATION        "Information"
```

That derivation lives in `features/inbox/utils/notificationType.ts`. Adding a new column to express the type would be the wrong fix; these two flags already carry it.

The actionable sub-modules

`notificationActionConfig.ts` is a registry of which sub-modules support a decision and how the reject dialog collects its remark:

SUB-MODULE	REJECT INPUT	NOTES
SHIFT_SWAP	Free text, required	Needs both a peer and a manager decision.
SHIFT_CHANGE	Free text, required	Manager decision.
LEAVE	Free text, required	Manager decision.
CAB_APPROVER	Reason picklist	Reasons come from the CAB reject-reason CRUD.
RESCHEDULE	Free text, required	Rewired in September 2026 to call the approve/reject procedures directly.

Adding a new actionable sub-module is deliberately two edits: one entry in this file, and one case in `useNotificationAction.ts`, which owns the RTK Query dispatch (the hooks must be called unconditionally at hook top level, so the mapping cannot live in a component). Nothing in the UI components changes.

Back-end endpoints

ENDPOINT	USE
GET <code>/notification/unread</code> , <code>/notificationcount</code>	The list and the sidebar badge.
POST <code>/notification/changedreadstatus</code>	Mark as read.
POST <code>/notification/swapreqempaction</code> / <code>swapreqmanageraction</code>	The two halves of a swap decision.
POST <code>/notification/shiftchangenotificationaction</code>	Shift change decision.
PATCH <code>/notification/leave-requests/{id}/status</code>	Leave decision.
POST <code>/notification/cabcrqnotificationaction</code>	CAB decision from the inbox.
POST <code>/notification/cabreschedulenotificationaction</code>	Reschedule decision.

Notification Manager

A separate screen — `/my-dashboard/notifications` , back end `/notification-manager` — where an administrator configures which roles are notified for which action, with per-role toggles and an `is_active` flag. It is gated on the “Notification Manager” sub-module of “Me”, which is the one deliberate tightening made when Home and Me were merged into My Dashboard.

21

Analytics, Data Agent, SFTP and Audit

Four smaller modules that are self-contained enough to learn in an afternoon each.

Analytics

Three tabs under `/analytics`, feature folder `crqAnalytics`:

- **Dashboard** and **CRQ Analytics** read `/crq-analytics-new/*`: a headline dashboard plus charts for circle/region, open domain, an aging heatmap, raised-versus-closed, run rate, KPI by stage and engineer utilisation. Every chart has a matching `/view-all/...` endpoint for its drill-down table.
- **Reports** is a file browser, not a query engine. It lists `.xlsx` files that an external process drops under `analytics.reports.base-path`, organised as `<base>/<type>/<date>/*.xlsx`, and serves them for download.

Charts are rendered with `chart.js` via `react-chartjs-2`. Use those, not a second charting library.

Data Agent

`/data-agent`. A natural-language-to-SQL chat and analytics surface, migrated in from a standalone application in August 2026 and rebuilt on the host app's chart library.

- Endpoints: `POST /dataagent/ask`, `POST /feedback`, and history management under `/history`.
- It calls out to a Python server whose URL comes from `PYTHON_SERVER_URL` in the external config file. Without that, the module compiles and renders but cannot answer.
- There is deliberately no `@PreAuthorize` on its controller — the authority source used elsewhere has no rows for this module, so annotating it would refuse every caller.

SFTP Management

`/sftp-management/windows` and `/linux`. Built in July 2026 as new functionality, role-exclusive to the Vegayan super-admin role.

- Windows tab → `/api/sftpfromui`: upload, list, download, delete against the configured local/remote path.
- Linux tab → `/api/sftp/linux/remote`: the same four operations against a remote Linux host over SFTP, using the maintained JSch fork. Note these are all `POST`, because the target host and directory travel in the body.

Building this module required fixing a pre-existing broken authority query as a prerequisite — the `@PreAuthorize` path had never worked. Be aware when adding annotations elsewhere.

Audit log

Covered mechanically in Chapter 9 and access-wise in Chapter 13. As a user-facing feature it is a filterable table at `/user-management/auditlog` showing who did what, in which module and sub-module, to which user, and when. The vocabulary is fixed: modules mirror `WEB_MODULE.module_name` so a row reads like the screen it happened on, and the verbs are a closed set (`CREATE` , `UPDATE` , `DELETE` , `APPROVE` , `REJECT` , `UPLOAD` , `DOWNLOAD` , `ASSIGN` , `UNASSIGN` , `ENABLE` , `DISABLE` , `SUBMIT` , `CANCEL` , `RESCHEDULE` , `LOGIN` , `LOGOUT`).

RULE OF THUMB FOR AUDITING YOUR OWN FEATURE

Annotate the endpoints that change state. Do not annotate a GET that merely renders a screen — opening a page is not a user action. Do annotate a GET that hands the user a document; that is a `DOWNLOAD` .

PART FIVE

Working in the Codebase



The conventions this team actually follows, one complete worked example of adding a feature across all three layers, the failure modes that have already cost days, and how the thing gets built and deployed.

-
- 22 Conventions — naming, structure, comments, commits
 - 23 Walkthrough — a new endpoint and screen, end to end
 - 24 Pitfalls and troubleshooting — the ones that look like other bugs
 - 25 Build and deployment

22

Conventions

Most of these are enforced by nothing except review. They are still worth following exactly, because every one of them exists because breaking it cost somebody a day.

Back end

- **Package by feature**, then `controller` / `service` / `dto`. A new functional area gets its own top-level package, not a folder inside an existing one.
- **Services extend `BaseService`** and use `databaseUtils`. Do not autowire a `JdbcTemplate` yourself.
- **No SQL in feature services**. A new read path is a new procedure and a new migration file.
- **Controllers return `ResponseEntity`**, and refusals use the typed exceptions so `GlobalExceptionHandler` produces a consistent body.
- **DTOs are Lombok classes or records**. Field names must match the underscore form of the procedure's column aliases (Chapter 10).
- **Version an endpoint rather than change its shape** when other callers exist — the codebase already has `/v1`, `/v2`, `/v3` variants for exactly this reason.

Front end

- **One folder per feature**, with the internal layout from Chapter 11. Export through `index.ts`; import other features only through their barrel.
- **Server state belongs to RTK Query**, not to a slice. Add a slice only for genuinely local UI state that several components share.
- **Never create a second `createApi`**. Always `api.injectEndpoints`.
- **Declare tags in `service/api.ts`** and match the existing spelling exactly, typos included.
- **Register a new route in the nav registry** even if it should not appear in the sidebar — use `ROUTE_ONLY_ACCESS_ENTRIES`. An unregistered path is *allowed* by the guard.
- **Lazy-load route elements**, and remember that a heavy library still lands in the `vendor` chunk unless `manualChunks` splits it.
- **TypeScript, no `any` in new code**. The existing code has some; do not add more.

THE COMMENT STYLE, AND WHY IT IS UNUSUAL

This codebase carries long explanatory comments where a decision is not obvious — often describing the bug that motivated the current shape. That is deliberate and it is the house style. When you fix something subtle, leave the *why* behind, not a restatement of the *what*. Several chapters of this handbook were reconstructed from exactly those comments.

Database migrations

Recapping Chapter 10 as a checklist, because this is where review is strictest:

- ☐ File is `db/migration/YYYY-MM-DD_what-it-does.sql`, dated when authored.
- ☐ Header comment states what already existed, what this adds, why, and how to roll back.
- ☐ `DROP PROCEDURE IF EXISTS` before `CREATE`; index creation guarded via `information_schema`. Running it twice is a no-op.
- ☐ Column aliases are the underscore form of the DTO field, with no underscore before a digit.
- ☐ A read procedure returns zero rows for “nothing found”, never an `error_message` column.
- ☐ Paging is `COUNT(*) OVER ()` in the same query, page size clamped inside.
- ☐ You dumped the deployed procedure and diffed it before editing.

Git

- Work on `master-main`; pull requests target `main`.
- Commit messages are short and feature-shaped — “Add MOP validation workspace and PDF viewer”.
- Ship the migration file in the same commit as the code that needs it. There is no separate change process for SQL.
- Remember the auto-commit watcher (Chapter 2): check `git log`, not `git status`, and re-check before assuming a change is unpushed.

23

Walkthrough: Adding a Feature End to End

A worked example. The task: add a screen listing every CRQ that has been paused for more than seven days, gated behind the Scheduler module. Six steps, three layers.

Step 1 — the procedure

New read path, so: new procedure, new migration file. Create `db/migration/2026-09-08_stalled_crq_report.sql`:

```
-- =====
-- Stalled CRQ report
-- Date   : 2026-09-08
-- Target : Vegayan_CHM (DBSOURCE1)
--
-- Existing and untouched: CRQ_MASTER_TBL, CRQ_STAGE_ASSIGN_TBL.
-- Adds      : Get_Stalled_CRQs – one paged read of CRQs sitting in ON_HOLD
--              for more than :p_days, scoped to a domain / sub-domain.
-- Why       : the CRQ Workflow list cannot express "paused for N days" and
--              planners were exporting to a spreadsheet to find them.
-- Rollback: DROP PROCEDURE IF EXISTS Get_Stalled_CRQs;
-- Re-runnable: yes (drop + create).
-- =====
DROP PROCEDURE IF EXISTS Get_Stalled_CRQs;
DELIMITER $$
CREATE PROCEDURE Get_Stalled_CRQs(
    IN p_domain_id    INT,
    IN p_sub_domain_id VARCHAR(10),
    IN p_days          INT,
    IN p_page          INT,
    IN p_size          INT
)
BEGIN
    DECLARE v_size INT DEFAULT LEAST(GREATEST(IFNULL(p_size, 25), 1), 200); -- clamp
    DECLARE v_off  INT DEFAULT GREATEST(IFNULL(p_page, 0), 0) * v_size;

    SELECT
        m.crq_no           AS crq_no,
        m.crq_id           AS crq_id,
        m.current_stage     AS current_stage,
        m.current_status    AS current_status,
        sa.stage_start_date AS paused_since,
        DATEDIFF(NOW(), sa.stage_start_date) AS paused_days,
        COUNT(*) OVER ()    AS total_count      -- page + true total, one trip
    FROM CRQ_MASTER_TBL m
    JOIN CRQ_STAGE_ASSIGN_TBL sa
        ON sa.crq_id = m.crq_id AND sa.stage = m.current_stage
    WHERE m.current_status = 'ON_HOLD'
        AND m.domain_id    = p_domain_id
        AND m.sub_domain_id = p_sub_domain_id
        AND DATEDIFF(NOW(), sa.stage_start_date) > IFNULL(p_days, 7)
    ORDER BY paused_days DESC
    LIMIT v_size OFFSET v_off;
END$$
DELIMITER ;
```

Note the three conventions in play: no `error_message` column (an empty result is simply no rows), `COUNT(*) OVER ()` for paging, and aliases that will bind cleanly to `crqNo`, `pausedSince`, `pausedDays`.

Step 2 — the DTO

```
package com.vegayan.airtelmanagement.scheduler.dto;

import lombok.Data;
import java.sql.Timestamp;

@Data
public class StalledCrqDto {
    private String    crqNo;           // ← crq_no
    private String    crqId;
    private String    currentStage;
    private String    currentStatus;
    private Timestamp pausedSince;     // ← paused_since
    private Integer   pausedDays;
    private Long      totalCount;
}
```

Step 3 — the service

```
@Service
public class StalledCrqService extends BaseService {

    public ProcedurePageResult<StalledCrqDto> findStalled(
        int domainId, String subDomainId, int days, int page, int size) {

        return databaseUtils.extractMultiPagedResult(
            jdbcTemplateTwo,           // DB2 – the CHM domain schema
            "CALL Get_Stalled_CRQs(?, ?, ?, ?, ?)",
            StalledCrqDto.class,
            domainId, subDomainId, days, page, size);
    }
}
```

Two decisions worth making consciously: `jdbcTemplateTwo` because CRQ data lives in DB2, and `extractMultiPagedResult` because the procedure pages.

Step 4 — the controller

```
@RestController
@RequestMapping("/crqworkflow")
@RequiredArgsConstructor
public class StalledCrqController {

    private final StalledCrqService service;

    @GetMapping("/stalled")
    public ResponseEntity<PageResponseDto<StalledCrqDto>> stalled(
        @RequestParam int domainId,
        @RequestParam String subDomainId,
        @RequestParam(defaultValue = "7") int days,
        @RequestParam(defaultValue = "0") int page,
        @RequestParam(defaultValue = "25") int size) {

        var result = service.findStalled(domainId, subDomainId, days, page, size);
        return ResponseEntity.ok(PageResponseDto.of(result, page, size));
    }
}
```

No `@Auditable` — this is a read that renders a screen. No `@PreAuthorize` either, unless the module has authority rows; the route guard and the org scoping already constrain it, and adding an annotation for a module with no authority rows refuses every caller (see the Data Agent note in Chapter 21).

Step 5 — the front-end API slice

```
// features/scheduler/api/stalledCrqApiSlice.ts
import { api } from "../../service/api";
import type { StalledCrq, StalledCrqParams } from "../../types/stalledCrq.types";

export const stalledCrqApiSlice = api.injectEndpoints({
  endpoints: (builder) => ({
    getStalledCrqs: builder.query<PagedResponse<StalledCrq>, StalledCrqParams>({
      query: ({ domainId, subDomainId, days, page, size }) => ({
        url: "/crqworkflow/stalled",
        method: "GET",
        params: { domainId, subDomainId, days, page, size },
      }),
      providesTags: ["StageWorkflow"], // declared in service/api.ts
    }),
  }),
});

export const { useGetStalledCrqsQuery } = stalledCrqApiSlice;
```

If you need a new tag, add it to `tagTypes` in `service/api.ts` first — RTK Query silently ignores an undeclared tag.

Step 6 — the page, the route and the grant

Three edits, in this order:

1. **The page.** `features/scheduler/page/StalledCrqMain.tsx`, reading the org scope from the shared filter hook and calling `useGetStalledCrqsQuery`.
2. **The route.** In `routes/AppRoutes.tsx`, a lazy import and a `<Route>` under the existing `scheduler` shell that is wrapped in `PrivateRoute`.
3. **The grant.** In `rbac/navRegistry.tsx`, a child under the Scheduler item:

```
{
  to: "/scheduler/stalled",
  text: "Stalled CRQs",
  icon: <HourglassEmptyOutlinedIcon />,
  requiredModule: "Scheduler",
  matchPaths: ["/scheduler/stalled"],
}
```

Gate at module level unless a matching `WEB_SUB_MODULE` row genuinely exists. Naming a sub-module that does not exist reads as “not granted” for every user — the page simply never appears, with no error anywhere.

VERIFY IN THIS ORDER

Run the `CALL` in a MySQL client → hit the endpoint with `curl` and a bearer token → check the JSON field names match your TypeScript type → open the screen. Each step rules out a whole layer, and the first two take under a minute. Restart the back end between steps 1 and 2 — there is no hot reload.

What you have not done, and should consider

- Nothing was audited — correct for a read.
- Nothing invalidates the new tag yet. If a stage action should refresh this list, add the tag to that mutation's `invalidatesTags`.
- The procedure is in the repository but not in the live database until somebody runs it. Say so explicitly in the PR description; this is the single most common cause of “works on my machine” in CHM.

24

Pitfalls and Troubleshooting

Every entry here is a real failure that has happened on this project, and every one of them initially looked like a different problem. Read the symptom column first.

Build and startup

Hundreds of “cannot find symbol” errors on getters and setters

Cause: not JDK 17. Lombok's annotation processing stops running and every `@Data` class loses its accessors.

Fix: point `JAVA_HOME` at JDK 17 and set the same SDK in IntelliJ's project settings and Maven runner.

The back end will not start and the stack trace mentions a properties file

Cause: the external config file is missing or unreadable at the hard-coded path.

Fix: create `C:\vegayan\simplus\airtelcms-config.properties` from the sample in `deployment/config/`. See Chapter 5.

The build fails on committed merge-conflict markers

It has happened that `<<<<<<` markers were committed into several source files, breaking every build until removed. If a fresh clone will not compile, grep for conflict markers before debugging anything else.

Authentication

Every request answers “Invalid session”

Cause: the `AUTH_JWT_TOKENS` row for this token's `jti` is gone, has `valid = false`, or has expired. It is never a malformed JWT — a bad signature or expiry fails earlier with a different message.

Fix: sign in again. If it recurs immediately, check whether something else is logging that account out: a second session elsewhere, or the cleanup job with a stale `expires_at`.

Login itself fails with “Invalid session”

Cause: historically, a stale token in the browser was validated by the JWT filter even on public routes, because `permitAll()` does not skip a filter. Fixed by sharing `PublicEndpoints.PATTERNS`.

If you see it again: you have probably added a public endpoint to `SecurityConfig` only. Add it to `PublicEndpoints` instead — both mechanisms read that one list.

“Already Logged” and you cannot get in

Use the terminate-session flow on the login screen (it re-checks your password), or delete the account's row from `AUTH_JWT_TOKENS`.

Data and procedures

A column is always `null` in the UI but correct in the database

Cause: the column alias does not match what `BeanPropertyRowMapper` derives from the DTO field. The classic case is a digit: `categorizationTier1` maps to `categorization_tier1`, never `categorization_tier_1`.

Fix: correct the alias in the procedure. There will be no error to guide you — only the null.

A search with no matches returns HTTP 500 and the screen freezes on old data

Cause: the read procedure reports “nothing found” as an `error_message` row, which `DatabaseUtils` turns into an exception and the API layer into a 500. RTK Query keeps the previous successful result on screen.

Fix: change the procedure to return zero rows. As a defensive measure, callers can catch a cause-less `DatabaseOperationException`.

A save reports success and nothing changes

Cause: either `updateUsingProcedure` was used on a procedure that can roll back, or the procedure has an empty `SQLEXCEPTION` handler that swallows the error and returns a success row. Both have happened.

Fix: use `executeProcedureForMessageV1`, and make the procedure report its own failure.

A procedure call fails with the wrong number of arguments

Cause: live/repository drift. Deployed procedures have gained and lost parameters without the `.sql` file changing.

Fix: `SHOW CREATE PROCEDURE` first, always.

A multi-result-set procedure suddenly returns the wrong data

Cause: row order or column names in `sp_get_crq_journey_page` have drifted twice already.

Fix: resolve rows by name, never by index.

Front end

“Invalid hook call” or “Cannot read properties of null (reading 'useContext')” in dev

Cause: two copies of React alive in one tab, caused by Vite's dependency optimiser re-running mid-session and bumping the shared `browserHash`. The stack trace shows two different `?v=` hashes. It is *never* a Rules-of-Hooks violation in your component.

Fix: stop the dev server, delete `node_modules/.vite`, restart, and do a **hard** reload of the tab. The `resolve.dedupe` and `optimizeDeps.include` entries in `vite.config.ts` exist to prevent this; do not remove them.

“Cannot access X before initialization” in a production build only

Cause: React, MUI and Emotion split into separate chunk files. Their circular internal references mean one chunk executes before another has finished initialising its exports.

Fix: keep them in the single `vendor` chunk. This is why `manualChunks` looks so blunt.

A lazy-loaded library still loads on app start

Cause: everything under `node_modules` goes to `vendor`, so `React.lazy` alone defers nothing.

Fix: give the library its own `manualChunks` branch, anchored on the package directory. Beware substring matching — `"react-pdf"` as a substring also catches `@react-pdf/renderer`, a different library.

Requests fail in the browser but work in `curl`

Cause: CORS. Your dev origin is not in the allow-list.

Fix: add it to `SecurityConfig.corsConfigurationSource()`.

A page is invisible for a role that should have it

Cause, in order of likelihood: the nav entry names the wrong module; it names a sub-module row that does not exist; the grant genuinely is not there.

Fix: compare the registry string against the live `WEB_MODULE` / `WEB_SUB_MODULE` rows, character for character.

A fit-to-container component flickers at certain window sizes

Cause: only `html` has `scrollbar-gutter: stable`, so a component that sizes itself to its container can oscillate across the scrollbar boundary — content fits, scrollbar disappears, content no longer fits, and round again.

Fix: write fit logic with hysteresis, or reserve the gutter on the container.

Verification without credentials

TWO HARNESSES THE TEAM HAS USED

When you need to check RBAC behaviour or take a screenshot but have no working login: build a small esbuild harness that imports the real `rbac/` modules and exercises the predicates directly; or drive the app with Playwright, intercepting `getloggeduserdetails` with a stubbed permission payload and seeding `localStorage`. Both avoid inventing a fake login path in the product.

25

Build and Deployment

Two artefacts, one Nginx site, one systemd unit or Windows service. The full runbooks live in `deployment/` in the front-end repository; this chapter is the shape of it and the parts that trip people.

The artefacts

ARTEFACT	BUILT BY	DEPLOYED TO
React bundle	<code>npm run ready-hoja</code> → <code>dist/</code>	<code>/var/www/chm/airtelchmbeta/</code> (Linux)
Spring Boot WAR	<code>mvnw package</code> → <code>target/changemanagement-test.war</code>	<code>/opt/chm/app/</code> , run with <code>java -jar</code>

The Nginx site

```
server {
    listen 80 default_server;
    root    /var/www/chm;

    client_max_body_size 1024m;    # Excel uploads, CRQ attachments, SFTP transfers
    proxy_read_timeout  300s;    # the backend triggers slow SSH/SFTP jobs

    location /airtelchmbeta/      { try_files $uri $uri/ /airtelchmbeta/index.html; }
    location /changemanagementnew/ { proxy_pass http://127.0.0.1:1857/; }
    location = /                  { return 302 /airtelchmbeta/; }
}
```

Three details matter. The SPA fallback to `index.html` is what makes a deep link survive a refresh. The large body size and long read timeout exist because real uploads and remote scripts are slow. Actuator on `8686` is deliberately *not* proxied.

Retargeting the base path

The SPA lives under a sub-path. To move it, change `VITE_APP_BASE_PATH` in `.env.production`, rebuild, and update the matching Nginx `location` blocks. Never edit source: Vite's `base` and `BrowserRouter`'s `basename` both derive from that one variable, so they cannot drift.

Linux specifics

Some Windows paths are baked into the WAR and must be overridden at launch — no rebuild needed:

```
java -jar /opt/chm/app/airtelmanagement.war \  
  --file.base.path=/opt/chm/files_path \  
  --file.base.path2=/opt/chm/files_path2 \  
  --logging.config=/opt/chm/config/logback-linux.xml \  
  --logging.level.com.vegayan.airtelmanagement=INFO
```

The last flag matters: `application.properties` ships several packages at `DEBUG`, which is far too noisy for production.

The external config file must exist at `/home/vegayan/simplus/config_airtel.properties`, and `/home/vegayan/simplus/sftp_uploads/` is a hard-coded working directory. The `deployment/scripts/linux/` folder carries `build.sh`, `deploy.sh`, start/stop/restart scripts, a systemd unit and a logrotate config; `deployment/scripts/windows/` has the equivalent `.bat` files and a service installer.

Verification after a deploy

- ☐ `curl http://127.0.0.1:8686/actuator/health` returns UP on the server.
- ☐ The login page loads at the base path and a real account can sign in.
- ☐ A deep link (for example `/airtelchmbeta/roster/view`) survives a browser refresh — this proves the SPA fallback.
- ☐ A file upload of realistic size completes — this proves the body-size and timeout settings.
- ☐ Any migration shipped in this release has actually been applied to the target database.

MIGRATIONS ARE NOT APPLIED BY DEPLOYMENT

Nothing in the build or start-up applies `db/migration/*.sql`. A release that adds or changes a procedure requires somebody to run that file against the target database. Make it an explicit line item in the release notes.

HTTPS

The deployment package includes `HTTPS-MIGRATION.md` and a reference `chm-airtel-https.conf`. The application itself needs no change — the JWT is stateless and travels in a header, so there is no sticky-session or cookie-path handling to revisit. HSTS should be added only once HTTPS is actually in place.

APPENDICES

Reference

Lookup material. Appendix A is the complete HTTP surface of the back end as it stands today — useful when you know the behaviour you want and need to find the code, or when you are checking whether something already exists before writing it.

A API endpoint reference — every controller, every route

B Directory map — both repositories

C Configuration reference — every key and where it lives

D First-week checklist

A

API Endpoint Reference

Grouped by controller. Paths are relative to the API base URL. This is a map of the code, not a contract — check the controller for parameter shapes.

Authentication and users

METHOD	PATH	NOTES
POST	<code>/auth/v1/signin</code>	Public. Returns <code>accessToken</code> + sets <code>jwt</code> cookie.
POST	<code>/auth/v1/logout</code>	Authenticated. Invalidates by <code>token_id</code> ; body <code>olmId</code> ignored.
POST	<code>/auth/v1/session/terminate</code>	Public but password-verified. Clears a stranded session.
POST	<code>/users/signup</code>	Public.
GET	<code>/users/v1 v2/getloggeduserdetails</code>	v2 is the one the app uses — profile + full RBAC hierarchy.
GET	<code>/users/getOrgHierarchyByUser</code> , <code>/users/V1/getOrgHierarchyByUser</code>	Org scope available to the caller.
GET	<code>/users/v3/getemployeesbysubdomain</code>	Employee picker data.
GET / POST	<code>/usermanagement/logindetails</code> , <code>/usermanagement/v1/getaccesstoken</code>	Login history; static integration token (public).
GET	<code>/audit-logs</code> , <code>/audit-logs/filters</code>	Super admin only, enforced server-side.

CRQ workflow — `/crqworkflow`

METHOD	PATH	NOTES
GET	<code>/overview</code> , <code>/overview/paged</code> , <code>/overview/{crqNo}</code>	The workflow list and one CRQ. <code>{crqNo}</code> is domain-scoped.
GET	<code>/search</code>	Global CRQ search, not org-scoped.
GET	<code>/cancelled</code> , <code>/cancelled/summary</code>	The cancelled register.
GET	<code>{crqNo}/plan-pdf</code>	Plan document.
GET / PUT	<code>/json/raw/{crqNo}</code>	Raw JSON payload for a CRQ (also on <code>/planningworkflow</code>).
GET	<code>/crqreview</code>	Plan & Inventory listing.

METHOD	PATH	NOTES
POST	<code>/updatecrqreview/{start pause done}</code>	Plan & Inventory transitions.
GET	<code>/impactanalysis</code> , <code>/impactanalysis/batch</code>	Listing; batch read (call <code>/impact/statuscsv/batch</code> first).
POST	<code>/updateimpactanalysis/{start pause done}</code> , <code>/impactanalysis-script</code>	Transitions; remote script trigger.
GET / POST	<code>/mopcreate</code> , <code>/mopcreate/{crqNo}/details</code> , <code>/mopcreate/{crqNo}/pdf</code>	MOP authoring; the PDF POST is multipart.
POST	<code>/updatemopcreate/{start pause done}</code>	MOP Create transitions.
GET	<code>/mopvalidate</code> , <code>/mopvalidate/{crqNo}/details</code> , <code>/mopvalidate/{crqNo}/workspace</code>	Review listing and workspace.
POST	<code>/mopvalidate/{crqNo}/review/start</code> , <code>/findings</code> , <code>/findings/{id}/state</code> , <code>/validate</code> , <code>/reject</code>	The review loop.
POST	<code>/updatemopvalidate/{start pause}</code>	MOP Validate transitions.
POST	<code>/refetchcheckpointscrip</code>	Re-pull checkpoint data over SSH.
GET / POST	<code>/validation/details</code> , <code>/validation/save</code>	The Plan & Inventory Validate dialog.
GET	<code>/journey-explorer/crqs</code> , <code>/crqNo</code> , <code>/crqNo/details</code>	CRQ Journey canvas data.

Reschedule — `/crq/reschedule`

METHOD	PATH	NOTES
GET	<code>/context</code> , <code>/reason-options</code>	Wizard step 1.
POST	<code>/initiate</code>	Opens a reschedule; returns a <code>rescheduleId</code> .
GET	<code>/rescheduleId/calendar</code> , <code>/rescheduleId/slots</code>	Candidate dates and slots.
POST	<code>/save-date</code> , <code>/confirm-slot</code> , <code>/move-stage</code> , <code>/cancel</code>	The remaining steps and the cancel action.

Plans, activities, tasks

METHOD	PATH	NOTES
GET	<code>/plan/view</code> , <code>/activity/view</code> , <code>/activity/phase-view</code>	Reads.
POST	<code>/activity/insert</code> , <code>/insertPlan</code> , <code>/updateplan</code> , <code>/phase-update</code>	Writes.
PATCH	<code>/activity/plan/{planId}/status</code> , <code>/activity/{activityId}/status</code>	Status toggles.
GET / POST	<code>/activity/excel/v1/{template parse upload}</code>	Plan + activity bulk upload (37 name-based parameters).
GET / PATCH	<code>/scheduler/task-config</code> , <code>/task-config/update</code>	Task configuration.
GET / POST	<code>/attributeupdate/details</code> , <code>/dropdown/*</code> , <code>/save</code>	The attribute-update dialog.

CAB — `/cab/*`

METHOD	PATH	NOTES
GET	<code>/cab/crqs</code> , <code>/mine</code> , <code>/services</code> , <code>{serviceApprovalId}</code> , <code>/mine/{serviceApprovalId}</code>	Listings and detail, keyed by service approval.
POST	<code>/cab/crqs/{id}/{approve reject delegate reschedule}</code>	The four decisions.
POST	<code>/cab/crqs/reschedule-requests/{crqNo}/approve</code>	Approve a reschedule request.
POST / GET	<code>/cab/crqs/{crqId}/assign-spoc</code> , <code>/assign-fe</code> , <code>{crqNo}/spoc-fe-details</code>	Ownership assignment.
GET / POST	<code>/cab/crqs/{crqNo}/conflicts</code> , <code>/conflicts/decision</code>	Conflict check.
GET / POST / PUT / DELETE	<code>/cab/crqs/cabrejectreasons</code>	Reject-reason CRUD.
GET	<code>/cab/crqs/service-csv/export</code>	Per-service CSV via a remote script.
GET	<code>/cab/crqs/implementation</code>	Execution tracking listing.
POST	<code>/cab/crqs/{crqId}/rings/{ringId}/{proceed block}</code>	Ring-level execution control.
GET / POST	<code>/cab/sessions</code> , <code>/cab/sessions/{sessionId}</code>	CAB sessions.
GET	<code>/cab/planning/queue</code> , <code>/dates</code> , <code>/cab/dashboard</code>	Planning and dashboard.
GET / POST	<code>/cab/admin/*</code>	Analytics, assign matrix, service rules, rejection reasons, escalation matrix, users, audit, dropdowns.

Roster, attendance and self-service

METHOD	PATH	NOTES
GET	<code>/monthlyroster</code> , <code>/userroster</code> , <code>/currentshiftcount</code> , <code>/shiftdropdowns</code>	<code>/userroster</code> is self-scoped.
POST	<code>/monthlyroster/changeshift</code> , <code>/shiftswapbymanager</code> , <code>/shiftswapreqbyteamember</code>	Shift writes.
GET / POST	<code>/goldenset</code> , <code>/goldenset/dailygoldenset</code>	Golden set.
GET / POST	<code>/rostergeneration/futureweek</code> , <code>/futureweek/update</code>	Note the spelling.
GET / POST	<code>/attendance/today</code> , <code>/workmode</code> , <code>/clockin</code> , <code>/clockout</code>	Dashboard attendance widget.
GET	<code>/dashboard/upcomingholidays</code> , <code>/employeesonleave</code> , <code>/dailyassignments</code> , <code>/worklocation</code>	Employee dashboard widgets.
GET / POST	<code>/leave/types</code> , <code>/leave/history</code> , <code>/leave/request</code>	Leave.
GET / POST / PATCH / DELETE	<code>/notification-manager/*</code>	Notification configuration.

Notifications — `/notification`

METHOD	PATH	NOTES
GET	<code>/unread</code> , <code>/notificationcount</code>	List and badge.
POST	<code>/changedreadstatus</code>	Mark read.
POST	<code>/swapreqempaction</code> , <code>/swapreqmanageraction</code> , <code>/shiftchangenotificationaction</code> , <code>/shiftchangestatuschange</code>	Roster decisions.
PATCH	<code>/leave-requests/{notificationId}/status</code>	Leave decision.
POST	<code>/cabcrqnotificationaction</code> , <code>/cabreschedulenotificationaction</code>	CAB and reschedule decisions.

Configuration

METHOD	PATH	NOTES
GET / POST / PATCH / DELETE	<code>/networkfreez/holiday/*</code> , <code>/networkfreez/networkfreeze*</code>	Holidays and freeze windows.
various	<code>/global-settings/permissions/*</code>	The RBAC editor — roles, modules, sub-modules, permissions, grants.
GET / POST / PUT / PATCH	<code>/global-settings/org-hierarchy/{verticals functions domains sub-domains}</code>	Org tree CRUD plus status.
GET / PUT / POST	<code>/teamoverview/*</code> , <code>/teamoverview/excel/v1/*</code>	Employees and bulk upload.

Analytics, integrations and files

METHOD	PATH	NOTES
GET	<code>/crq-analytics-new/*</code>	Dashboard, per-chart data, and a <code>/view-all/...</code> drill-down for each.
GET	<code>/analytics/reports/{type}</code> , <code>/type/{date}</code> , <code>/type/{date}/download/{fileName}</code>	Generated report file browser.
POST / GET / DELETE	<code>/dataagent/ask</code> , <code>/feedback</code> , <code>/history*</code>	Natural-language analytics.
POST	<code>/api/remedy/update</code> , <code>/cab-request/*</code> , <code>/change-request/planning-in-progress</code> , <code>/planning/v1/submit</code> , <code>/planning/v1/slot_crq_check</code>	Outbound Remedy integration.
GET / POST	<code>/remedycygnetsync/queue</code> , <code>/run</code> , <code>/mark-done/{crqNo}</code>	Cygnetsync control.
POST	<code>/api/slots/viewslot</code> , <code>/bookslot</code> , <code>/api/calendar/view</code> , <code>/crq_status/update/{crqNo}</code>	Slot booking and status push.
GET	<code>/impact/statuscsv/batch</code> ; POST <code>/excel/impact-batchwise</code>	Impact batch listing and Excel generation.
POST / GET / DELETE	<code>/api/sftpfromui/*</code>	Windows SFTP: upload, list, download, delete.
POST	<code>/api/sftp/linux/remote/*</code>	Linux SFTP — all POST, target travels in the body.
GET	<code>/test/hello</code>	Public reachability check.

B

Directory Map

Where things are, in both repositories.

Front end — CHM_airtel_beta

```
CHM_airtel_beta/
├── .env .env.local .env.production  API base URL and deployment base path
├── vite.config.ts                  dedupe, optimizeDeps, manualChunks, dev server
├── index.html  package.json  tsconfig*.json  eslint.config.js
├── deployment/
│   ├── README.md  LINUX-DEPLOYMENT.md  HTTPS-MIGRATION.md  INSTRUCTIONS-HINGLISH.md
│   ├── nginx/{windows,linux,https}/    site configs
│   ├── scripts/{windows,linux}/        build, deploy, service install, start/stop
│   └── config/                          airtelcms-config.sample.properties, logback-linux.xml
└── src/
    ├── main.tsx  App.tsx  index.css
    ├── app/      store.ts, hooks.ts, store/{auth.storage,postLoginRedirect}.ts, config/
    ├── service/  api.ts ← the single RTK Query instance
    ├── routes/   AppRoutes, PrivateRoute, PublicRoute, DefaultRedirect, LegacyRedirect
    ├── rbac/     navRegistry.tsx, routeAccess.ts, permissionCore.ts, usePermission.ts,
    │            useSidebarNav.tsx, PermissionGate.tsx, AccessDenied.tsx
    ├── components/ layout/ (AppLayout, Header, SideBar), common/, ui/, loading/
    ├── style/     MUI theme and colour mode
    └── features/  auth, myDashboard, dashboard, userMe, scheduler, crqJourney,
                  cabManager, roster, rosterGenerator, teamManagement, userManagement,
                  settings, orgHierarchy, inbox, rescheduleNotification, crqAnalytics,
                  dataAgent, sftpManagement, globalSearch
```

Back end — airtelmanagement

```
airtelmanagement/
├── pom.xml  mvnw  mvnw.cmd
├── db/
│   ├── README.md                  the migration conventions – read it
│   ├── migration/                 ~45 dated .sql files
│   └── scheduler-stage-history/    supporting scripts
└── src/main/
    ├── resources/application.properties  ports, JWT, Remedy, Cygnet, logging
    ├── resources/logback-spring.xml
    └── java/com/vegayan/airtelmanagement/
        ├── AirtelmanagementApplication.java  ServletInitializer.java
        ├── common/{config,security,service,util,dto,exception}
        ├── auth  user  usermanagement  teammanagement
        ├── scheduler  cabmanager  remedy  remedycygnetsync  sygnet  impactbatch
        ├── rosterview  rostergeneration  attendance  me  notification
        ├── globalsettings  orghierarchy  activity  attributeupdate
        ├── analyticsreport  crqanalytic  dataagent  dashboard
        └── audit  filetransfer
```

Files worth reading on day one

FILE	WHY
<code>src/rbac/navRegistry.tsx</code>	The whole product's navigation and gating in one file.
<code>src/service/api.ts</code>	Every HTTP concern the front end has.
<code>src/routes/AppRoutes.tsx</code>	Every screen, in order.
<code>common/util/DatabaseUtils.java</code>	Every way the back end talks to the database.
<code>common/security/jwt/JwtAuthenticationFilter.java</code>	Everything about who a request is.
<code>db/README.md</code>	The rules you will be reviewed against.
<code>features/scheduler/constants/stageConfig.ts</code>	The clearest example of the codebase's config-driven style.



Configuration Reference

Four places configuration lives. Knowing which one a value comes from is usually the whole answer.

1. External properties file — secrets and hosts

`C:\vegayan\simplus\airtelcms-config.properties` (Windows) or `/home/vegayan/simplus/config_airtel.properties` (Linux). Read by `AppPropertiesConfig` at construction; the application will not start without it.

KEY	MEANING
<code>DBSOURCE_USERMGMT_{IP,USER,PASS,DBNAME}</code>	DB1 — user management schema.
<code>DBSOURCE1_{IP,USER,PASS,DBNAME,PORT}</code>	DB2 — CHM domain schema.
<code>SSH_{HOST,PORT,USERNAME,PASSWORD}</code>	Primary script host.
<code>SSH_SCRIPT_PATH</code>	General-purpose script.
<code>SSH_IMPACT_ANALYSIS_SCRIPT_PATH</code>	Impact-analysis script.
<code>SSH_REFETCH_CHECKPOINT_SCRIPT_PATH</code>	Checkpoint refetch.
<code>SSH_SERVICE_CSV_SCRIPT_PATH</code>	Per-service CSV for the CAB export. Defaults if absent.
<code>SSH_SHIFT_GEN_VAL_SCRIPT_PATH</code>	Shift-generation validation.
<code>SSH_PDF_VIEW_DOWNLOAD_PATH</code>	Where PDFs are fetched from.
<code>SSH2_*</code>	A second script host with the same shape.
<code>SFTP_{HOST,PORT,USERNAME,PASSWORD,FILE_PATH}</code>	SFTP endpoint and base path.
<code>SFTP_JSON_RAW_FILE_PATH</code>	Raw CRQ JSON documents.
<code>SFTP_FILE_PATH_CRQ_WEORKLOG_FILE</code>	CRQ work logs. (Spelling is as-is.)
<code>SFTP_FILE_PATH_BATCH_CSV_FILE</code>	Impact batch CSVs.
<code>SFTP_BATCHWISE_IMPACT_EXCEL_PATH</code>	Generated impact Excel files.
<code>PYTHON_SERVER_URL</code>	The Data Agent's Python backend.

Defaults exist in code for the local SFTP working directory (`...\simplus\sftp_uploads`) and the Linux SFTP remote directory (`/tmp`).

2. `application.properties` — committed, environment-agnostic

See the table in Chapter 5. Summary of the important ones: `server.port=1857`, `management.server.port=8686`, `jwt.token.expirationTime=604800000`, `jwt.secret`, `auth.master.password`, `remedy.env` and the paired SIT/PROD URLs, credentials and API keys, `remedy.cygnet-sync.*`, `spring.task.scheduling.pool-size=3`, `spring.servlet.multipart.max-file-size=100MB`, `analytics.reports.base-path`.

3. Front-end env files

VARIABLE	MEANING
<code>VITE_REACT_APP_BASE_URL</code>	API base URL. Read via <code>app/config/env.ts</code> and by <code>service/api.ts</code> .
<code>VITE_APP_BASE_PATH</code>	Deployment sub-path. Feeds Vite <code>base</code> and the router <code>basename</code> .
<code>VITE_CAB_USE MOCK</code>	Whether CAB screens read mock data. <code>false</code> everywhere today.

4. Runtime data — configuration that lives in the database

Easy to forget, and the cause of many “it works for me” differences:

- RBAC grants in the `WEB_*` tables — who sees what.
- The organisation hierarchy — what rows anyone can see.
- Holidays and network-freeze windows — when work can be scheduled.
- Notification configuration — who is told about what.
- CAB rejection reasons, assignment and escalation matrices.

Ports at a glance

PORT	SERVICE
1857	Spring Boot API.
8686	Actuator. Loopback only; never proxied.
5174	Vite dev server (bound to <code>0.0.0.0</code>).
80 / 443	Nginx in a deployed environment.
3306	MySQL.

D

First-Week Checklist

If you can tick everything here by Friday, you are ready to pick up a real ticket.

Day 1 — running

- ☐ Both repositories cloned under `D:\CHM_airtel_scheduler\Github_Repo\`.
- ☐ `java -version` reports 17. `JAVA_HOME` and IntelliJ agree.
- ☐ External config file created; the back end starts and `/actuator/health` is UP.
- ☐ `.env.local` written; `npm run chalu-hoja` serves the login page.
- ☐ You can sign in, and you know how to clear an “Already Logged” session.

Day 2 — orientation

- ☐ Visited every screen your role can open; you can name the module behind each.
- ☐ Read `src/rbac/navRegistry.tsx` top to bottom.
- ☐ Read `src/service/api.ts` and understand the 401/403 handling.
- ☐ Opened a CRQ and traced its seven stages in the UI.
- ☐ Read `db/README.md`.

Day 3 — tracing

- ☐ Picked one screen and followed a request from the browser network tab to the controller, to the service, to the `CALL` in the SQL log.
- ☐ Ran that `CALL` yourself in a MySQL client and matched the output to the screen.
- ☐ Dumped one deployed procedure with `SHOW CREATE PROCEDURE` and diffed it against its migration file.
- ☐ Found where that screen's route, nav entry and permission gate are declared.

Day 4 — a small change

- ☐ Made a visible change to one screen and saw it hot-reload.
- ☐ Made a back-end change, restarted, and confirmed it — remembering there is no hot reload.
- ☐ Added a field end to end: procedure alias → DTO → TypeScript type → UI.
- ☐ Confirmed the alias binds (no silent `null`).

Day 5 — understanding the traps

- ☐ You can explain why “Invalid session” is always about a database row.
- ☐ You can explain why a read procedure must not return `error_message`.
- ☐ You can explain why `updateUsingProcedure` can report a false success.
- ☐ You can explain why the live database and `db/migration/` may disagree, and what to do about it.
- ☐ You know that hiding a sidebar item is not a security boundary.
- ☐ You know that a migration in the repository has not been applied to any database until somebody runs it.

Questions worth asking your team

- Which database am I pointed at, and who else is using it?
- Which role code am I signed in as, and which one should I test against?
- Which migrations in `db/migration/` are *not* applied to my environment?
- Is the Cygnet sync meant to be on in this environment?
- Who owns the Remedy integration credentials?

ONE LAST PIECE OF ADVICE

When something in CHM behaves oddly, the answer is usually in one of three places: the SQL log (what was actually asked of the database), the live procedure body (what actually ran), or the nav registry (what the user was actually allowed to see). Check those three before you reach for a debugger.